



DetectVul: A statement-level code vulnerability detection for Python

Hoai-Chau Tran^{a,c}, Anh-Duy Tran^d, Kim-Hung Le^{b,c,*}

^a University of Science Ho Chi Minh City, Viet Nam

^b University of Information Technology, Ho Chi Minh City, Viet Nam

^c Vietnam National University, Ho Chi Minh City, Viet Nam

^d imec-DistriNet, KU Leuven, Leuven, Belgium

ARTICLE INFO

Keywords:

Source code vulnerability detection

Deep learning

Natural language processing

ABSTRACT

Detecting vulnerabilities in source code using graph neural networks (GNN) has gained significant attention in recent years. However, the detection performance of these approaches relies highly on the graph structure, and constructing meaningful graphs is expensive. Moreover, they often operate at a coarse level of granularity (such as function-level), which limits their applicability to other scripting languages like Python and their effectiveness in identifying vulnerabilities. To address these limitations, we propose DetectVul, a new approach that accurately detects vulnerable patterns in Python source code at the statement level. DetectVul applies self-attention to directly learn patterns and interactions between statements in a raw Python function; thus, it eliminates the complicated graph extraction process without sacrificing model performance. In addition, the information about each type of statement is also leveraged to enhance the model's detection accuracy. In our experiments, we used two datasets, CVEFixes and Vudenc, with 211,317 Python statements in 21,571 functions from real-world projects on GitHub, covering seven vulnerability types. Our experiments show that DetectVul outperforms GNN-based models using control flow graphs, achieving the best F1 score of 74.47%, which is 25.45% and 18.05% higher than the best GCN and GAT models, respectively.

1. Introduction

Code vulnerability detection is an important task in modern software development as it helps ensure the safety and security of software users. Attackers can compromise the integrity of applications or systems by exploiting vulnerabilities embedded in source code [1]. To detect potential security vulnerabilities early in the development process, source code vulnerability detection (SVD) is often performed as part of static application security testing (SAST). Traditional SVD approaches using rule-based methods such as SonarQube,¹ Bandit,² and Pylint³ can be error-prone due to the complexity and variability of real-world scenarios [2]. In recent years, text-based and graph-based approaches using deep learning models have shown promise in adapting to changes in real-world projects and enabling broad coverage of vulnerability detection. However, to the best of our knowledge, most previous works that used text-based or graph-based approaches only conducted their research on C/C++ due to its rich ecosystem of tools [3–7] to preprocess and extract meaningful data (i.e. graphs structures, low-level information, etc.), while language like Python was ignored despite its

popularity and widespread usage across areas like artificial intelligence, web development, automation, system administration, IoT, and game development.

In recent years, text-based approaches have been widely used to detect vulnerabilities, and Vudenc [8] is one typical example that was specifically built for Python. Specifically, in Vudenc, the author employed an RNN-based architecture like LSTM to directly identify the vulnerability in a chunk of Python code with a fixed window size. However, text-based approaches are limited by their focus on a high level of granularity, such as function-level [9–13], slice-level [8,14–17], whereas a vulnerable pattern can exist in just a few lines of code. Specifically, in text-based approaches like Vudenc, the SVD task is typically treated as a text classification problem, where the input code snippet, often a function, is converted to a plain text sequence. This sequence is normalized and tokenized with NLP techniques before being fed into deep learning models like RNN-based [18,19] or BERT-based models [20,21] for classification. There are already many studies that used this text-based approach for C/C++ [9–17].

Interpreting input code as plain text sequences often results in the loss of crucial structural information between code statements. For

* Corresponding author at: University of Information Technology, Ho Chi Minh City, Viet Nam.

E-mail addresses: thchau18@apcs.fitus.edu.vn (H.-C. Tran), anh-duy.tran@kuleuven.be (A.-D. Tran), hunglk@uit.edu.vn (K.-H. Le).

¹ <https://docs.sonarqube.org/>

² <https://github.com/PyCQA/bandit>

³ <https://pypi.org/project/pylint/>

statement-level SVD, detecting vulnerabilities by treating statements as isolated plain text is challenging without the surrounding context, often leading to numerous false positives (*i.e.*, low precision) in predictions. For instance, without context, it is difficult to determine whether line 2 in Fig. 2 and line 4 in Table 2 are vulnerable, as we cannot verify whether the inputs in these statements are properly sanitized in previous statements or how the outputs are used in subsequent ones. While predicting at higher levels of granularity (slice-level, function-level, file-level) provides the model with richer context and may improve accuracy, it also leads to longer review times and increased effort to identify and fix vulnerable statements. Additionally, text-based approaches are often constrained by a fixed context window (*e.g.*, Vudenc [8] processes slices with a maximum of 200 tokens, and Transformer-based models [20,21] have a 512-token context window). This limitation can result in the model failing to fully capture the logic of code blocks or causing corrupted statements when the context window truncates a long statement.

To overcome the limitations of text-based approaches, GNN-based methods have emerged as a promising solution, achieving state-of-the-art results in C/C++ SVD in recent years. Typically, previous works [22–27] leveraging GNN models often include three main components: (i) A data preprocessing pipeline to construct graphs, (ii) a feature extractor, often a pretrained static model [28,29] or language model [21] to encode nodes and produce embeddings vectors, and finally (iii) a GNN-based classifier [30,31] to learn node interactions and provide node-level (statement-level) predictions. Given that C/C++ allows direct memory manipulation, which often leads to vulnerabilities like buffer overflows and memory leaks, previous works [22,27] have constructed graphs such as program dependency graphs (PDGs) and code property graphs (CPGs) to uncover data-related vulnerabilities. Control flow graphs (CFGs) and call graphs (CGs) have also been used to trace execution paths where input validation may be bypassed [23, 25,26]. This GNN-based approach can also be adapted for Python, where models trained on CFGs can analyze and identify Python-specific vulnerabilities, typically involving improper input validation, insecure deserialization, or misuse of security-sensitive functions, rather than memory corruption.

However graph-based approaches have two major limitations. Firstly, the performance of GNN-based model relies heavily on the quality of the graph structure, and constructing meaningful graphs such as CPGs or PDGs, which provide richer context by capturing syntax structures, control flow, and data dependencies in a unified graph, can be expensive in terms of both time and space complexity. Secondly, the inductive biases of graph models, such as local connectivity and predefined graph structures, limit their ability to identify more complex patterns and learn from distanced statements due to missed connections. To overcome these limitations, previous studies have either increased the number of layers in the GNN model or combined different graph structures to capture more features from distant code statements. The latter strategy is often preferred in previous studies [9,22–24,32] as adding more layers to GNN architectures can result in over-smoothing and over-squashing issues [33,34], potentially degrading the model's performance. On the other hand, adapting the graph construction process from one programming language to another can be challenging. For instance, Well-known tools such as Joern [7] and LLVM [3] are often used to build CPG or DPG, but they are tailored for languages like C/C++ and do not yet support Python. This limitation poses a challenge to the scalability of GNN-based approaches.

This paper aims to introduce a novel architecture designed to avoid the need for graph extraction while still achieving superior performance on the statement-level SVD task for Python. To do this, in the proposed architecture, we eliminate the dependency on graphs by replacing the GNN model with a flexible BERT-like architecture to learn the complex relationships between statements in code snippets or functions. In detail, the model is structured with two BERT-based models, serving as a feature extractor and a classifier. The motivation behind this idea

is that the architecture of the transformer's encoder block is a specific case of graph neural network (GNN), where nodes are globally interconnected [35], and the self-attention layer in each encoder block enables forming data-driven graphs without the need for an explicitly defined graph structure. This is similar to the message-passing mechanism of GNNs [36], where trainable weighted connections are established between every pair of nodes in the graph. Moreover, this approach frees the detection model from predefined graph structures, enabling it to leverage large amounts of data from realworld open-source projects to form more meaningful structural information between nodes, leading to better performance. Furthermore, given that vulnerabilities appear exclusively in specific types of statements, such as conditions, assignments, or loops, we also incorporated information about each statement type to enhance the model's ability to identify and address vulnerabilities accurately (see Fig. 1).

The training data for the proposed model is obtained from two publicly available datasets, CVEFixes [37] and VUDENC [8], which originated from GitHub commits used to patch issues related to vulnerabilities in different open-source projects. To make the model more robust to adversarial attacks, all user-defined names, such as variables and functions, are normalized into symbolic names (*e.g.*, VAR_1, VAR_2, and FUNC_1), and statements are extracted and labeled based on changes in collected commits from open-source projects on GitHub. Our data processing and labeling criteria are also introduced to reduce duplicated samples and false positive ground truth labels, which are further discussed in Section 4.2. In summary, this paper makes the following contributions:

- We propose DetectVul, a simple and effective model that could perform statement-level SVD for Python source code. Using the self-attention mechanism in the transformer encoder, the model could learn connections and relationships among statements in a function without using any predefined graph structure. The information about each statement type is also incorporated to increase detection accuracy. More information about the data processing, model architecture, and training strategy is available in Section 4.
- Several experiments were conducted to benchmark the performance of DetectVul and GNN-based baseline models. To the best of our knowledge, no previous work has constructed a graph dataset for Python. Hence, we have created our own datasets from VUDENC and CVEFixes datasets using control flow graphs (CFGs). The empirical result on these datasets shows that DetectVul outperforms baselines using GNN-based models. Details about the main research questions and the empirical results are presented in Section 5.
- To further advance the field and enable future research, we have made our implementations including source code for the whole pipeline (data preprocessing, training, and inference), datasets,^{4,5} and model checkpoints publicly available on GitHub and HuggingfaceHub. This allows researchers to reproduce our experiments and build upon our work. Moreover, an additional dataset for C/C++ that has labeled statements⁶ based on the famous Devign dataset [10] is also released to help future researchers benchmark our work on C/C++.

2. Related works

This section provides an overview of deep learning approaches for SVD in recent years (from 2020 to 2023), to help position our work in the larger context of the field. We focus mainly on two perspectives:

⁴ Vudenc:<https://huggingface.co/datasets/DetectVul/Vudenc>

⁵ CVEFixes:<https://huggingface.co/datasets/DetectVul/CVEFixes>

⁶ Devign:<https://huggingface.co/datasets/DetectVul/devign>

```

105         if import_from_settings('OIDC_USE_NONCE', True):
106             nonce = get_random_string(
107                 import_from_settings('OIDC_NONCE_SIZE', 32)
108             )
109             params.update({
110                 'nonce': nonce
111             })
112             request.session['oidc_nonce'] = nonce
113
114             request.session['oidc_state'] = state
115             request.session['oidc_login_next'] = (
116                 request.GET
117                 .get(redirect_field_name)
118             )
119
120             query = urlencode(params)
121             redirect_url = (
122                 '{url}?{query}'
123                 .format(
124                     url=self.OIDC_OP_AUTH_ENDPOINT,
125                     query=query
126                 )
127             )
128             return HttpResponseRedirect(redirect_url)
129

```

Fig. 1. Our model has identified a code sample that is susceptible to CWE-601 (URL Redirection to Untrusted Site) vulnerability, with a stronger indication of vulnerability represented by a darker red line. Here, statements that lie between lines 115 to 118 assign the value of the 'redirect_field_name' parameter received in the request to the 'oidc_login_next' session variable, which is then used later to redirect the user after the authentication process is complete. If the value of 'redirect_field_name' is controlled by an attacker, they can potentially set it to a malicious URL and trick the user into visiting it.

Table 1

A summary of the different deep learning approaches used in recent years (from 2020 to 2023).

Name	Dataset	Feature extractor & Classifier	Model type	Level of granularity	Language
VulDeeLocator (2020) [16]	SARD [38] + NVD [39]	Word2vec [28] + BGRU [19]	Sequence	Slice	C/C++
VUDENC (2022) [8]	VUDENC [8]	Word2vec [28] + LSTM [18]	Sequence	Slice	Python
Reveal (2020) [9]	Reveal [9]	Word2vec [28] + GGNN [40]	Graph	Function	C/C++
FUNDED (2020) [26]	SARD [38] + NVD [39]	Word2vec [28] + GGNN [40]	Graph	Function	C/C++, Java, PHP
DeepWukong (2021) [32]	SARD [38] + Redis + Lua	Doc2vec [29] + GCN/GAT [30,31]	Graph	Function	C/C++
IVdetect (2021) [24]	SARD [38] + NVD [39]	GloVe [41] + FA-GCN [24]	Graph	Statement	C/C++
LineVD (2022) [23]	Big-Vul [42]	CodeBERT [21] + GAT/GCN [30,31]	Graph	Statement	C/C++
MVD (2022) [22]	MVD [22]	Doc2vec [29] + FS-GNN [22]	Graph	Statement	C/C++
LineVul(2022) [43]	Big-Vul [42]	CodeBERT [21] + MLP	BERT	Statement	C/C++
CAG(2022) [27]	CAG [27]	6 different GNNs	Graph	Function	C/C++, Java
vulchecker(2023) [25]	vulchecker [25]	Structure2Vec [44] + MLP	Graph	Statement	C/C++
DetectVul (This paper)	VUDENC [8] + CVEFixes [37]	BERT variants [21,45,46] + BERT [20]	BERT	Statement	Python

The architecture of the deep learning model and the level of granularity of the predicted result (e.g., function-level, statement-level). See Table 1.

2.1. Sequence-based models

Sequence-based/RNN-based models, such as LSTM [18], GRU [19], and bidirectional variants have been extensively used for analyzing sequence or time-series data. In the domain of detecting vulnerabilities in source code, the data is typically partitioned into chunks/slices of text where these slices are tokenized into a sequence of tokens and then fed to an RNN-based model. Since most previous works [8,14–17] for slice-level SVD are performed using the same kind of architecture (LSTM, GRU) and embedding method. Typically, static embedding techniques, such as Word2vec [28], are often employed to extract representation vectors, which are then fed into an RNN-based model (e.g., BiLSTM, BiGRU) for classification. Therefore, many approaches have a data-centric methodology, where different approaches are used in the data preprocessing steps to enhance the final performance of the model. In recent years, VulDeeLocator [16] is one approach that took the sequence-based approach by leveraging program analysis and deep learning techniques to generate SeVCs and remove false positives by extracting tokens from the source code based on SyVCs. The intermediate code is used to identify semantically related statements, which are encoded into vectors and then used to train the model to perform classification.

As far as we know, VUDENC [8] is the only work that has developed a solution for detecting vulnerabilities in Python code. Particularly, in VUDENC, the author tokenized and extracted overlapping code slices,

where each slice consists of 200 tokens. After that, Word2vec and LSTM are employed for feature extraction and classification.

2.2. Graphs-based models

Recent years have witnessed a rise in the popularity of GNN-based approaches for detecting vulnerable patterns in source code. These methods use different graph structures with statements as nodes, thereby enabling the combination of feature vectors of each statement and its neighbors. Unlike slice-level methods, which consider feature vectors in isolated slices, GNN-based methods explicitly train feature vectors of each statement combined with contextual features such as edges and neighbor nodes, thereby potentially enhancing the model's ability to detect vulnerable patterns in a wider context. To the best of our knowledge, GNN-based methods are currently considered the dominant approach in this field and have been found to achieve state-of-the-art results. Here, we only mention some of the most notable studies

For function-level vulnerable code detection, there are approaches such as Devign [10] and Reveal [9], that focus on detecting vulnerabilities at the function level. These approaches utilize Word2vec in conjunction with gated graph recurrent layers (GGNN) [40] to generate embedding vectors from various graph structures, including AST, CPG, CFG, DDG, and natural code sequences (NCS). A study called FUNDED [26], took one step further by encoding the control-flow and data-flow information between statements and performing trainable aggregation operators based on each relationship, which enriched the useful information in the embedding vector of each line of code. DeepWukong [32] is another approach for detecting bugs in software

programs, which first constructs a PDG based on control flow and data flow information. It then generates slices from the PDG using forward and backward traversal starting from a program point of interest. During the detection phase, control and data dependencies of a target program are extracted, symbolized, and embedded by Doc2vec [29] to feed into trained GNNs like GCN [30] or GAT [31] models for graph-level classification. In addition, a recent study by Luo et al. [27] introduced a new type of graph called Compact Abstract graphs (CAG). Given a C/C++ or Java function, an abstract graph is constructed, after that, all single-entry node sequences and aggregation structures are merged to form a CAG. The empirical results show that CAG can outperform other types of graphs like CFG, PDG, CG.

Recently, there has been an increasing focus on the use of GNNs to achieve statement-level classification for identifying software vulnerabilities. One prominent example of this approach is IVDetect [24], which uses a PDG to differentiate vulnerable code from surrounding contextual code. IVDetect then employs a GCN [30] with feature-attention (FA-GCN) to perform graph-level classification. Then, GNNExplainer [47] is utilized to identify crucial program dependencies and statements that are most relevant to the detected vulnerability. Another model, LineVD [23], takes a single function of source code as input and processes it into individual statements. This study uses CodeBERT [21] to separately embed both function-level and statement-level codes and focuses on data and control dependency information. These features are then fed into GAT or GCN for training and inference. Moreover, MVD [22] is another method for memory-related vulnerability detection that constructs a PDG with additional semantic information from a CG and reduces noise by conducting program slicing. The statements of each slice are transformed into low-dimensional vector representations using Doc2vec [29], and a new architecture called flow-sensitive graph neural network (FS-GNN) is used to learn vulnerability patterns. One of the most recent studies that can detect vulnerabilities at statement-level is vulchecker [25], in which the author leverages LLVM compiler to create enriched program dependency graphs (ePDG) and employs Structure2Vec (S2V) [44], a message-passing Graph Neural Network (GNN), for effective information propagation while considering edge features. Additionally, VulChecker employs a cost-effective data augmentation method, injecting synthetic traces from open-source labeled datasets to enhance generalization in real code analysis.

2.3. Transformer-based models

In addition to GNN-based methods, another emerging approach for detecting vulnerabilities at the statement level involves employing self-attention scores within the Transformers architecture. This approach closely aligns with the methodology presented in our paper. A notable prior study employing this method is LineVul [43]. Particularly, LineVul utilized pretrained models such as CodeBERT to extract feature vectors from code tokens and added a simple linear layer for function-level classification. The attention scores assigned to each token were then used to assess each line's contribution to the final prediction, effectively pinpointing vulnerable lines. Our approach, DetectVul, takes one step further by directly treating each line as an individual token. We perform line-level classification using a dedicated BERT model. By pooling line-level embedding vectors and conducting classification for all lines, our method offers greater flexibility where the model is not restricted by the number of tokens within a function, in contrast to LineVul, where the model can only operate for functions with fewer than 512 tokens.

3. Problem formation

In this paper, the classification of vulnerable statements is treated as a multiclass classification problem where each class is one type of vulnerability. Given a source file, all statements are extracted and grouped into function gadgets, where each gadget represents a set of

statements that belong to a single function. Our dataset can be denoted as $\mathbf{D} = \{\mathbf{d}_1, \mathbf{d}_2, \dots, \mathbf{d}_N\}$, where N is the number of gadgets. Let $\mathbf{s}_i$ and $\bar{\mathbf{y}}_i$ represent the set of statements of function gadget $\mathbf{d}_i$ and their corresponding labels, we have $\mathbf{d}_i = (\mathbf{s}_i, \bar{\mathbf{y}}_i) = ((s_{ij}, y_{ij}) | s_{ij} \in \mathbf{s}_i, y_{ij} \in \bar{\mathbf{y}}_i, y_{ij} \in \{0, 1, 2, \dots, c-1\}, j \in \{1, 2, \dots, n\})$, in which c denotes the number of classes and n is the number of statements in one function. A function gadget $\mathbf{d}_i$, depicted in Fig. 2, is a processed function wherein each statement s_{ij} are normalized and has their trailing spaces, tab, and newline escape removed. The statements in $\mathbf{s}_i$ are arranged in the same order as they appear in the original function. The vector $\bar{\mathbf{y}}_i$ contains the labels y_{ij} that correspond to each statement s_{ij} .

Let f and θ be the deep-learning model and the trainable weights that are used to map $\mathbf{s}_i \rightarrow \bar{\mathbf{y}}_i$, we have $\hat{\mathbf{y}}_i = f(\mathbf{s}_i, \theta)$, where $\hat{\mathbf{y}}_i \in \mathbb{R}^{n \times c}$ is a matrix that denotes predicted probability distributions of each label for all statements in $\mathbf{s}_i$. Our final goal is to find the set of weights θ^* that can help approximate $\hat{\mathbf{y}}_i$ to $\bar{\mathbf{y}}_i$ by minimizing a loss function $L(\theta)$, the whole learning process can be formulated as:

$$\underset{\theta}{\operatorname{argmin}} L(\theta) = \sum_{i=1}^N \sum_{j=1}^n \frac{l_{ij}(f(s_{ij}, \theta), y_{ij})}{\sum_{i=1}^n w_{y_{ij}}} \quad (1)$$

Here, given that the dataset is often imbalanced — where vulnerable statements make up only a small portion of the codebase — we need to apply a heavier penalty for misclassifying vulnerabilities. This is achieved using a weighted cross-entropy (WCE) loss function. Here in (1), l_{ij} denotes the weighted cross-entropy (WCE) loss function, and $w_{y_{ij}}$ is the weight assigned to each statement based on its ground truth label y_{ij} .

4. Methodology

This section presents the overview of our methodology including key stages such as data preprocessing, training, and inference.

4.1. Overview

Our approach involves three main phases: preprocessing data, training model, and detecting vulnerable codes (model inference), as illustrated in Fig. 3.:

First phase: we extract and normalize function gadgets from Python files via three steps:

- Firstly, we extract all user-defined names from the input file. Next, all statements are extracted and labeled based on vulnerable code patches found on GitHub. These patches consist of a series of commits that update code to resolve open security-related issues. If a code snippet was altered in a security-related commit, we assume that the modified part of the code was previously vulnerable.
- The processed statements are then grouped into function gadgets in the same order as they appear in the original function.
- Finally, we employ code normalization, replacing all user-defined names saved from the first step with symbolic names. The output of the first phase is a collection of N function gadgets, denoted as $\mathbf{D} = \{\mathbf{d}_1, \mathbf{d}_2, \dots, \mathbf{d}_N\}$, where each function gadget $\mathbf{d}_i = (\mathbf{s}_i, \bar{\mathbf{y}}_i)$ consists of n normalized statements $\mathbf{s}_i = \{s_{i1}, s_{i2}, \dots, s_{in}\}$ and a vector $\bar{\mathbf{y}}_i = \{y_{i1}, y_{i2}, \dots, y_{in}\}$ containing corresponding labels.

Most steps in phase 1 involve using the AST module. The reason that we chose AST as the main tool to preprocess data is due to its availability in many programming languages (e.g. Java, Javascript, C/C++, etc.), which also makes it easier for future researchers to adapt our pipeline to other languages.

Second phase: we feed all statements in a function gadget $\mathbf{d}_i$ into the model. Note that, unlike previous approaches [8,14,17,23] that used pretrained feature extractors and only trained the GNN-based classifier model, in our study we integrated both of them into a single

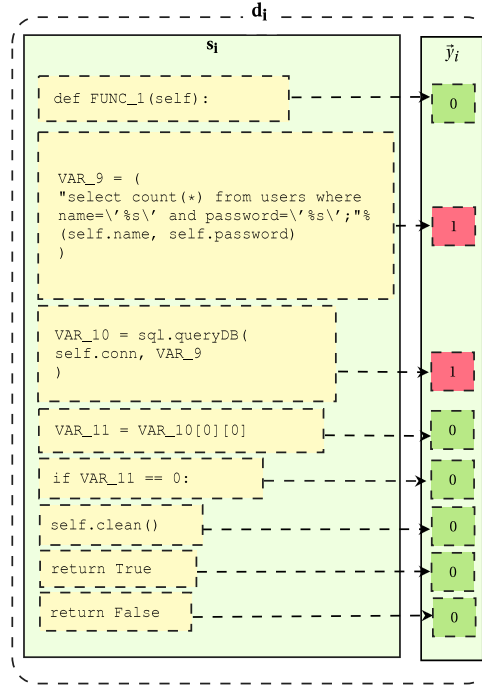


Fig. 2. An illustration of a function gadget $d_i \in \mathbf{D}$ that consists of 8 statements $s_{ij} \in s_i$ and their corresponding labels $y_{ij} \in \bar{y}_i$. All statements have had their trailing spaces, tab, and newline escape removed. Here, two of the statements, namely s_{i2} , and s_{i3} , are susceptible to SQL injection attacks. An attacker can exploit this vulnerability by manipulating the input parameters 'self.name' and 'self.password' to insert malicious SQL code into the query.

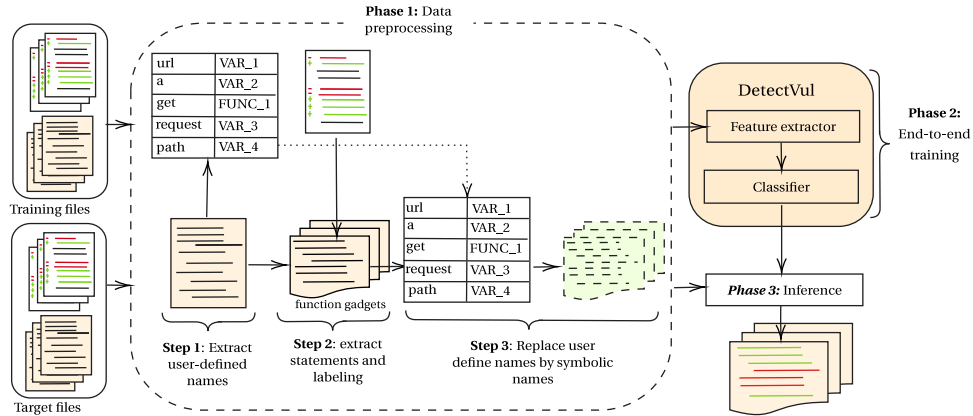


Fig. 3. DetectVul pipeline overview.

model and had the model trained end-to-end. This training strategy was used for all baselines and new models. As illustrated in Fig. 4, given a collection of statements s_i , the feature extractor model extracts the feature embeddings on the fly and feeds them to the classifier model to classify non-vulnerable and vulnerable code statements. The final output is a matrix $\hat{Y}_i \in \mathbb{R}^{n \times c}$ that contains the probability distribution of all classes for each statement.

Third phase: we use the well-trained model to conduct inference and evaluation on function gadgets. The evaluation process is carried out in diverse scenarios, which includes testing the performance of the DetectVul model on different hyperparameter settings and comparing it with other GNN-based architectures such as GAT [31] and GCN [30]. Furthermore, experiments to test the model's ability to withstand adversarial attacks are also conducted by performing the SVD task on the original and the augmented versions of the evaluating dataset, in which all user-defined methods and variable names are replaced by random strings.

4.2. Data preprocessing and labeling

4.2.1. Code normalizing

Given a Python source file, to ensure that the model has access to the relevant feature required to provide accurate predictions, we need to normalize the source code by eliminating any inconsistencies such as variables, input parameters, and function names. These unpredictable names, commonly referred to as noise, pose a challenge for deep-learning-based approaches as they prevent the models from generalizing to new, unseen data.

Specifically in step 1, when a Python file is parsed by the AST module, the AST object is generated to represent the source code as a tree structure of nodes. Each node in the tree corresponds to a syntactic element in the program, such as a variable declaration or an if statement, and is labeled with a syntax category like "If", "FunctionDef" or "Assign". In our study, we utilized this information to extract user-defined names from code components that create or assign values to variables or define new functions, while ignoring components

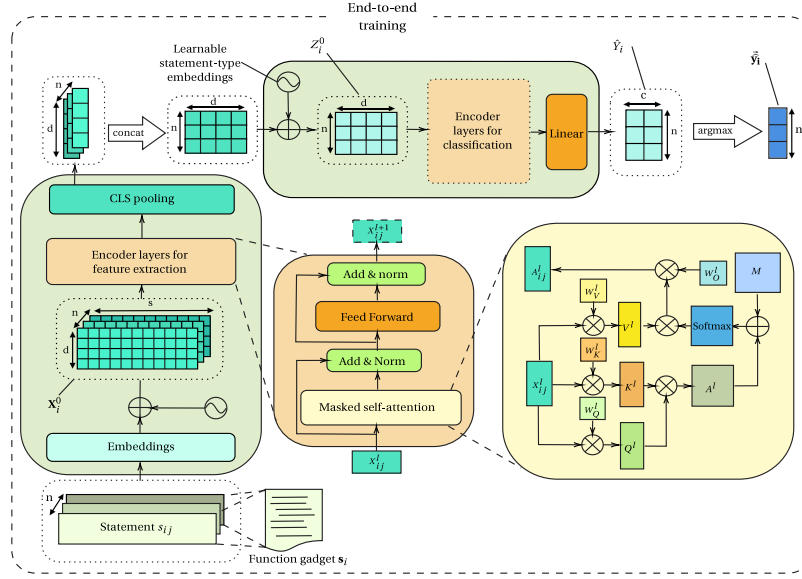


Fig. 4. The DetectVul training pipeline treats each function gadget as a batch of statements. Given a function gadget s_i , the statements are tokenized and then fed into an embedding layer and added to positional embedding matrices, which produces an initial embedding tensor $X_i^0 \in \mathbb{R}^{n \times d}$. Here, n represents the number of statements in the input gadget, s is the length of the longest statement, and d is the hidden dimension of each embedding vector. The feature vectors are then concatenated and added to another learnable statement type embedding (STE) matrix to form a feature matrix for all statements $Z_i^0 \in \mathbb{R}^{d \times n}$ to serve the classifier models. In the classification stage, the statement features are processed through another series of encoders to learn about relationships between statements. The final output goes through a softmax function, resulting in an output matrix $\hat{Y}_i \in \mathbb{R}^{n \times c}$, where c is the number of classes. The predicted result is then obtained by selecting the class with the highest probability.

that represent API or function calls. For example, let us consider this Python function:

```
def calculate_sum(a, b):
    result = a + lib.api_call(b)
    return result

total = calculate_sum(5, 10)
```

When the code is parsed by the AST module, an AST object is generated, with each node representing a syntactic element. For example, the line “def calculate_sum(a, b):” is represented by a “FunctionDef” node, while both “result = a + b” and “total = calculate_sum(5, 10)” are represented by “Assign” nodes. This information is used to label the type of each statement. Additionally, user-defined names, such as the function name “calculate_sum” and the variable names “result” and “total”, “a”, “b”, are extracted, while API call like “lib.api_call()” is ignored.

The collected names are stored in a Python dictionary, which serves as a reference for step 3. Since in step 2, each statement is labeled by its line number before the normalization process, this two-step normalization is essential to maintain the original line numbers during labeling. Further details about the labeling process will be provided in the next section. In step 3, all variable names and their occurrences throughout the source files are replaced with generic names. For example, after step 3, the previous sample code snippet will become:

```
def FUNC_1(VAR_1, VAR_2):
    VAR_3 = VAR_1 + lib.api_call(VAR_2)
    return VAR_3

VAR_4 = FUNC_1(5, 10)
```

By normalizing the data, the model can be prevented from memorizing unnecessary syntax features and enhance the robustness and reliability in predicting vulnerable patterns.

4.2.2. Statements extracting and labeling

While extracting all functions from the prior code version of each commit, to minimize the duplication rate in the dataset, in the updated

code version, we only consider functions that contain new/modified lines. All docstrings within each function are also removed as they are related to vulnerable patterns and may negatively impact the performance of the model. To process a function into a gadget, it is first divided into individual statements. The processed statements are then stripped of all tabs, trailing spaces, and new line escapes and reassembled in their original order to form a function gadget. To label a statement as vulnerable, the information about starting and ending line numbers is retrieved to check if any line that contains vulnerable code, falls within this span. Additionally, unlike other datasets that contain only functions [9,10], all import statements are extracted, as there may be an unsafe or corrupted module imported into the code, these import lines are also grouped and treated as a function gadget. A demonstration of a function gadget is illustrated in Fig. 2.

In the labeling process, since manual labeling is not feasible due to the lack of domain knowledge and limited resources, deleted lines in each commit are leveraged as a basis for identifying vulnerabilities. However, there might be many potential false positives in the ground truth labels, since many commits are not directly related to the vulnerability or only resolve a portion of the problem. Also, changes in the code may not be directly related to the vulnerability.

To help better clean the data, more criteria are applied to label a changed line as vulnerable or not. First, to reduce the duplication rate in the original datasets [8], only the last commit per issue is exploited, as there are cases when developers commit multiple times to patch the vulnerability before closing one issue. Second, in a commit, if the deleted lines contain only empty lines or comments, it can be safely assumed that the line is not vulnerable. On the other hand, given a deleted line with valid content, if there is no identical line present in the added lines we can keep it in the vulnerable lines list. The intuition behind this approach is based on our assumption that there are cases where the deleted lines themselves are not actually vulnerable but when combined with a specific context they form a vulnerable pattern. For example, in Table 2, although the entire function was modified in the commit, only line number 4 of the deleted version is susceptible to CWE-22 (path traversal). By applying the criteria outlined above during the labeling process, we can effectively reduce the number of

false positives, thereby minimizing noise in the data. Moreover, the developer can also simply change the line number without changing the content of the line (e.g., in Table 2 line 6 of deleted version is the same as line 8 of added version).

4.3. DetectVul

Architectures such as BERT [20] and its variants [48–51] have gained a lot of popularity in Natural Language Understanding (NLU) tasks, including token classification, sequence classification, and extractive question-answering. Similarly, these models can also be applied to the field of programming languages to learn complex ideas behind the syntax. As illustrated in Fig. 4, the architecture of DetectVul consists of two distinct BERT-based models, each with a specific mission. The first model acts as a feature extractor to provide contextualized dynamic embeddings from statements in a function gadget, whereas the second model is tasked with learning the relationships between those feature vectors to produce final predictions.

4.3.1. Feature extractor

The chosen architecture for the feature extractor (FE) utilizes BERT [20] architecture to output dynamic contextualized embedding instead of traditional static embedding methods [28,29,41,52]. When it comes to detecting vulnerabilities in Python, we can benefit from pre-trained models that have already been trained on a large corpus of Python code. A recent empirical study [53] also demonstrated that the use of BERT for extracting representation vectors for Python consistently outperforms other static embedding methods, such as Word2Vec [28] and fastText [52], across almost all experiments for vulnerable code detection. Moreover, there are various models available [21,45,46] that utilize diverse unsupervised learning techniques to comprehend the relationships between code tokens in different programming languages. Some of these models have been trained in multiple languages, making it easy to expand our approach to other programming languages in the future. Details about the pre-trained models that are used as the feature extractor are available in Table 4.

As denoted in Section 4.1, given a function gadget $\mathbf{d}_i = (\mathbf{s}_i, \mathbf{y}_i)$ in the dataset $\mathbf{D}$, we have $\mathbf{s}_i = \{s_{i1}, s_{i2}, \dots, s_{in}\}$ represent all statements and $\mathbf{y}_i = \{y_{i1}, y_{i2}, \dots, y_{in}\}$ denotes the labels associated with each statement. Before feeding the statements into the representation learning model, they are initially broken down into tokens using the Wordpiece tokenization algorithm [54]. The statement preprocessing stage can be represented by the following formula:

$$\mathbf{X}^0 = \text{Embedding}(\text{Tokenize}(\mathbf{s}_i)) + PE \quad (2)$$

In (2), the resulting arrays of tokens are then fed into an embedding layer to produce an input embedding matrix for each statement. To incorporate positional information into the model, the fixed positional encoding method was employed. Let PE be the generated positional embedding matrix that is added to the input embedding matrix for each statement. The final output of (2) is the embedding tensor $\mathbf{X}_i^0 = \{X_{i1}^0, X_{i2}^0, \dots, X_{in}^0\} \in \mathbb{R}^{n \times s \times d}$ with n as the number of statements in each gadget, s as the longest statement length, and d as the hidden dimension of each embedding vector.

As illustrated in Fig. 4, the input embedding tensor $\mathbf{X}^0$ is fed to a series of encoders using a multihead self-attention operator to extract the representation vector of each statement. For simplification, the formula of the self-attention layer is formulated with a single head. Let the $\mathbf{X}_i^l$ embedding tensor be the input of the l th encoder block, for each $X_{ij}^l \in \mathbf{X}_i^l$ with $j \in \{1, 2, 3, \dots, n\}$, n is the number of statements in the current function gadget, we have:

$$\text{MaskedAttention}(Q, K, V) = \text{Softmax}\left(\frac{QK^T}{\sqrt{d_k}} + M\right)V \quad (3)$$

$$A_{ij}^l = \text{MaskedAttention}(X_{ij}^l W_Q^l, X_{ij}^l W_K^l, X_{ij}^l W_V^l) W_O^l \quad (4)$$

$$X_{ij}^{l+1} = \text{LayerNorm}(\text{MLP}(\text{LayerNorm}(A_{ij}^l + X_{ij}^l))) + A_{ij}^l \quad (5)$$

Eqs. (3), (4) and (5) contain three primary components: ‘MaskedAttention’, MLP (multi-layer perceptron), and ‘LayerNorm’. Here, the ‘MaskedAttention’ function refers to the self-attention layer that involves several sets of trainable weights denoted as W_Q^l , W_K^l , W_V^l , and W_O^l , each with a size of $d \times d$. A ‘Softmax’ function is used to normalize attention scores into a probability distribution. To prevent certain tokens, such as the symbolic names ‘VAR_2’, ‘VAR_3’, and ‘VAR_4’, from being attended to by other tokens, we apply an attention mask matrix, denoted as M that has the same size as the attention score matrix, the two matrices are then added together to obtain the final attention scores (see Fig. 5). The ‘MLP’ function represents a fully connected feed-forward neural network, whereas ‘LayerNorm’ uses the layer normalization technique [55] to normalize the layer values to ensure consistent and well-scaled data.

Finally, after passing through all the layers of the encoder in the feature extractor model, the resulting embedding tensor $\mathbf{X}_i^l$ is sent through a pooling layer to extract the final representation vector for each statement. This is achieved by selecting the first token embedding of each matrix X_{ij}^l (e.g., the ‘[CLS]’ token) as the representation vector of a whole statement. Let us denote the output of the last encoder layer as l^* , we have:

$$\bar{z}_{ij} = \text{CLSPooling}(X_{ij}^{l^*}) \quad (6)$$

Each embedding vector $\bar{z}_{ij}$ is then used as an input to the classifier model, discussed in the following section.

4.3.2. Classifier

The learning process in the classifier model follows the same pattern as that of the feature extractor. Since we now have representation vectors for each statement from the feature extractor model, we directly concatenate them and feed them to the classifier to produce the final prediction output.

$$Z^0 = \parallel_{j=1}^n \bar{z}_{ij} + STE \quad (7)$$

$$\text{Attention}(Q, K, V) = \text{Softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (8)$$

$$A_i^l = \text{Attention}(Z_i^l W_Q^l, Z_i^l W_K^l, Z_i^l W_V^l) W_O^l \quad (9)$$

$$Z_i^{l+1} = \text{LayerNorm}(\text{MLP}(\text{LayerNorm}(A_i^l + Z_i^l))) + A_i^l \quad (10)$$

In (7), $\parallel$ is the concatenate operation and STE (Statement Type Embeddings) is a learnable embedding module that was used to provide embedding vectors for each statement type. The input for the classifier model is an embedding matrix $Z_i^0 \in \mathbb{R}^{n \times d}$ representing the embedding matrix for the function gadget s_i . This matrix undergoes another series of encoders, similar to the feature extractor, but this time all statements are attended to without any mask, as shown in (8). Let l^* denote the final encoder layer, we have:

$$\hat{Y}_i = \text{Linear}(Z_i^{l^*}) = W^C Z_i^{l^*} \quad (11)$$

In (11), W^C is the trainable weight matrix of size $d \times c$ that is used to project embedding matrix $Z_i^{l^*}$ to the matrix $\hat{Y}_i \in \mathbb{R}^{n \times c}$ where c is the number of classes. For each j in $\{1, 2, \dots, n\}$ and k in $\{0, 2, \dots, c-1\}$, we can get the predicted labels $\hat{y}_{ij}$ for all statements s_i by choosing the class index k with the highest values in each vector $\hat{y}_{ij} \in \hat{Y}_i$.

$$\hat{y}_{ij} = \parallel_{j=1}^n \arg\max_k (\hat{y}_{ijk}) \quad (12)$$

4.3.3. End-to-end training

In the real-world scenario, our primary objective is to pinpoint specific code statements that are potentially suspicious and require the developer’s attention for further investigation. Therefore, it is more important for a model to accurately identify all statements that contain vulnerable code, rather than avoiding false positive predictions. To accomplish this, we assign weights to all statements in the dataset and use the weighted cross-entropy (WCE) loss to train the model.

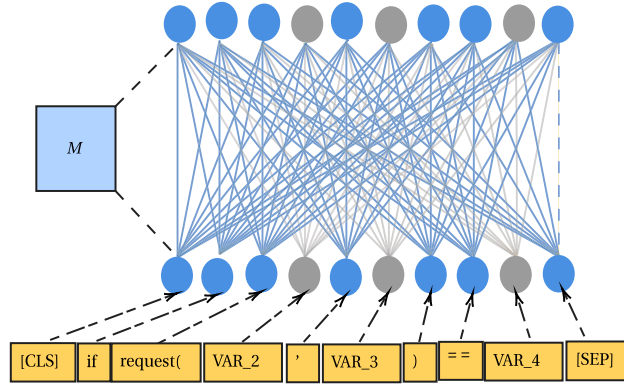


Fig. 5. At every self-attention layer of each encoder block of the feature extractor, to emphasize the features that contribute to vulnerable patterns, all attention weights related to symbolic names, which are defined in the first step of the data preprocessing phase, are filtered out. This is done by adding a mask matrix M to the attention matrix A .

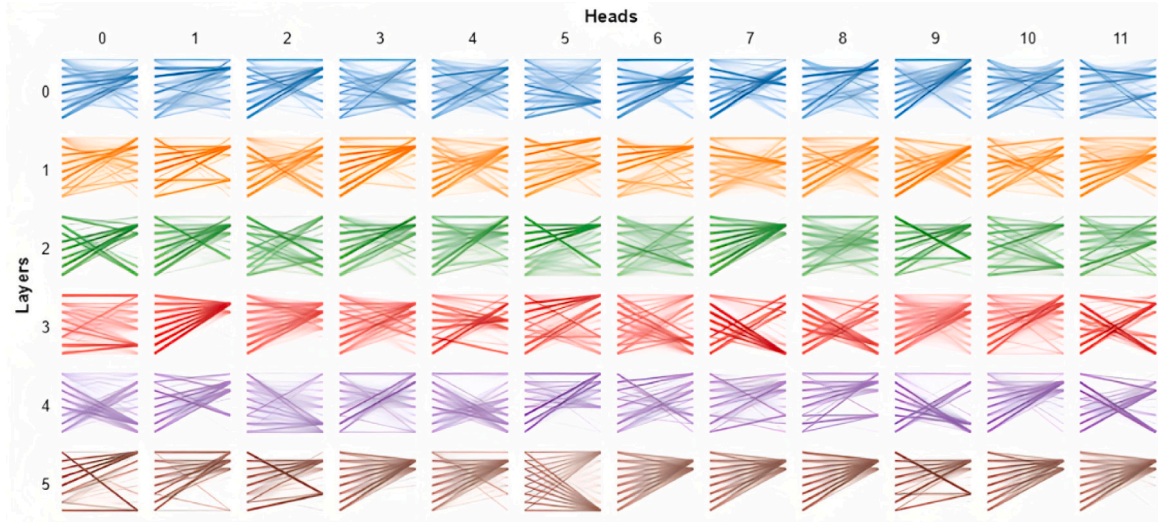


Fig. 6. BERTViz [56] is used to visualize attention patterns for each head in a classifier model that contains 6 encoder layers, represented by columns and rows respectively. Here bolder lines imply higher attention weights. Each cell contains a thumbnail version for each head of the following Fig. 7, which is specific to the input statements s_i of function gadget d_i , depicted in Fig. 2.

Let us denote θ as the parameters of the deep learning model f . For each function gadget $d_i = (s_i, \tilde{y}_i)$ that have n statements, the loss function is:

$$L_i(\theta) = \sum_{j=1}^n \frac{l_{ij}(f(s_{ij}, \theta), y_{ij})}{\sum_{i=1}^n w_{y_{ij}}} = \sum_{j=1}^n \frac{l_{ij}(\tilde{y}_{ij}, y_{ij})}{\sum_{i=1}^n w_{y_{ij}}} \quad (13)$$

$$l_{ij} = -w_{y_{ij}} \log \left(\frac{\exp(\hat{y}_{ij}(y_{ij}))}{\sum_{k=0}^{c-1} \exp(\hat{y}_{ij}(k))} \right) \quad (14)$$

In (13), the loss function $L_i(\theta)$ is defined as the weighted sum of all statement losses. During the training process, we used the adaptive gradient algorithm (Adam) [57] with a learning rate of 1×10^{-5} . The model's parameters are updated based on one function gadget with batch size of n statements for each step. Let us $w_{y_{ij}} \in \{w_0, w_1, w_2, \dots, w_{c-1}\}$ denotes the weight assigned to each statement s_{ij} , which is calculated based on the number of statements that have the ground truth label y_{ij} , and the total number of statements in the dataset.

$$w_{y_{ij}} = \frac{s}{c \times s_{y_{ij}}} \quad (15)$$

Here in (15), s is the total number of statements in all functions gadgets, c is the number of classes, and $s_{y_{ij}}$ is the number of statements that have label y_{ij} .

4.4. Model explanation

To understand how DetectVul's classifier model identifies vulnerable statements, we examine a use case involving the function gadget shown in Fig. 2. This is a Python function that contains statements that are susceptible to SQL injection attacks. Specifically, statements 2 and 3 can be exploited by an attacker who manipulates the input parameters 'self.name' and 'self.password' to insert malicious input into the SQL query expression. Fig. 6 illustrates a classifier model of DetectVul that has six layers of transformers encoder.

Taking a closer look at the way the conditional statement s_{i5} focuses on other statements in Fig. 7, we can see that most of the attention heads can recognize the two most important statements in the function gadget (statements s_{i2} and s_{i3}), which contain the SQL statement and are also the center of data-flow and control-flow in this function. Other heads, such as the red, brown, and purple heads, tend to pay attention to the two statements (s_{i6} and s_{i7}) that are influenced by the current conditional statement. From Fig. 6 it is clear that by stacking multiple encoder layers that utilize multi-head self-attention mechanisms, the DetectVul model can form various attention patterns in each head, which allows it to cover a wider range of relationships between statements than GNN-based approaches with predefined graph structures.

In Fig. 6, it is also noteworthy that at every attention head, as the contextualized embeddings go to the deeper layer, the attention

Table 2

The example of Github commit is used to secure file upload functionality from CWE-22 (Path Traversal) vulnerability.

Deleted	Added
<pre> 1 - def upload_file(file, upload_dir): 2 - filename = file.filename 3 - 4 - file_path = os.path.join(upload_dir, 5 - filename) #Vulnerable line (path traversal) 6 - file.save(file_path) </pre>	<pre> 1 + def upload_file(file, upload_dir): 2 + filename = file.filename 3 + # Remove potentially dangerous 4 + # characters and limit length 5 + file_path = os.path.join(6 + upload_dir, 7 + re.sub(r'[^a-zA-Z0-9_.-]', '_', 8 + filename)[:255] 9 +) 10 + file.save(file_path) </pre>

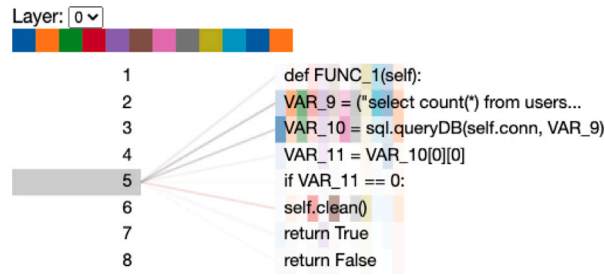


Fig. 7. A visualization generated by BERTViz [56] that shows how the conditional statement s_{15} gathers contextualized information from other statements.

patterns seem to just focus on the two specific statements which are also statements that contain the vulnerability. According to a previous study conducted by Clark et al. [58], this phenomenon also means that the models cannot find anything else to attend to and choose to bypass the current self-attention layer by choosing to gather the same embedding features as the previous layers. From this intuition, we can easily see that all heavy-lifting jobs are actually done in the first few layers, and increasing more encoder layers in the classifier models is not necessary. In fact, this might also lead to the overfitting problem which is demonstrated in empirical results in Section 5.5.4.

5. Experiments

This section shows DetectVul's evaluation results corresponding to our research questions. It also introduces the datasets, evaluation metrics, and hyperparameters employed in the experiments.

5.1. Research questions

This study aims to evaluate the effectiveness of DetectVul in the SVD task by addressing the following research questions (RQs):

- RQ1: How much do code normalization and attention masking contribute to DetectVul's capability of detecting vulnerable code?
- RQ2: How does DetectVul compare with baseline GNN architectures in terms of statement classification accuracy?
- RQ3: What is the effect of statement type embeddings on DetectVul?

5.2. Metrics for evaluation

Vulnerable code usually accounts for a small fraction of the code-base, resulting in highly imbalanced datasets for vulnerable code detection. Therefore, we adopted F1 score, precision, and recall as the evaluation metrics for the classification model, this is also a common

practice used in related studies [22,23]. As our model performs multiclass classification tasks and the ground truth labels are imbalanced, so macro F1, macro precision, macro recall, and Receiver Operating Characteristic Curve (AUROC) are used as the evaluation metrics.

$$Precision_{macro} = \frac{1}{c} \sum_{i=1}^{c-1} \frac{TP_i}{TP_i + FP_i} \quad (16)$$

$$Recall_{macro} = \frac{1}{c} \sum_{i=1}^{c-1} \frac{TP_i}{TP_i + FN_i} \quad (17)$$

$$F1_{macro} = \frac{2 \times Precision_{macro} \times Recall_{macro}}{Precision_{macro} + Recall_{macro}} \quad (18)$$

$$AUROC = \int_0^1 TPR(FPR) dFPR \quad (19)$$

The results of the $TPR(FPR)$ function in (19) describe the trade-off between TPR and FPR at various classification thresholds and is often plotted as ROC. The AUROC is then calculated as the area under this curve.

5.3. Datasets

In this paper, we used two datasets to evaluate DetectVul's performance: VUDENC [8] and CVEFixes datasets [37]. These datasets are collected from GitHub commits in open-source projects and are publicly available. The VUDENC dataset, in particular, includes GitHub commits that tackle problems associated with specific common weakness enumerations (CWEs) in open-source repositories. However, it has a primary limitation which is the substantial number of duplicate samples, which can cause severe data leakage problems between the train and test sets and result in over-optimistic experimental results. Therefore to better evaluate our models, the CVEFixes dataset is employed, which was created by collecting common vulnerabilities and exposures (CVE) records from the National Vulnerability Database (NVD) and identifying relevant GitHub commits that addressed the vulnerabilities via the attached URLs that are linked to the patching commits. Table 3 provides the number of function gadget samples for each CWE.

Table 3The statistics about the number of function gadgets (d_i) for each vulnerability type in the evaluated datasets.

Dataset	Normal	CWE-22	CWE-77	CWE-79	CWE-89	CWE-352	CWE-601	CWE-94	Total no. of function gadgets	Total no. of statements	Avg statements per function	Avg statement length
CVEFixes	5301	68	n/a	102	n/a	40	150	69	15 841	153 919	9.72	11.79
VUDENC	13 020	325	328	58	1123	526	246	215	5730	57 398	10.02	13.22

Table 4

Pretrained feature extractor models.

FE	Num layers	Hidden size	Pooling	Model size
MiniLM [46]	12	384	CLS pooling	120 MB
mpnet [45]	12	768	CLS pooling	420 MB

5.4. Pretrained feature extractors

In our experiments, two different embedding models, mpnet [45] and MiniLM [46], were utilized as feature extractor models for all baselines and new models in our experiment. The mpnet model has a larger hidden size of 768, and MiniLM has a smaller hidden size of 384. Both models were pretrained using contrastive learning objective [59] on a massive corpus of over one billion sentence pairs, including code in multiple programming languages from the CodeSearchNet dataset [60]. Table 4 provides additional information about these feature extractors such as the number of encoder layers, the pooling strategy, and the model size.

5.5. Experiment results

This section describes the experimental setup and results for each of the research questions outlined in Section 5.1. Insights derived from the empirical results are also discussed to answer these questions.

5.5.1. Experiment results for RQ1

This experiment evaluates the impact of the code normalization steps and the attention mask M on the performance of DetectVul.

Data preparation: We create two versions of evaluated datasets (VUDENC and CVEFixes) for this experiment. Particularly, to demonstrate the robustness of our method to adversarial attack, we created the first dataset version which contained unprocessed function gadgets with the original code, and the second version replaced all inconsistencies (e.g., user-defined variable names, method names, and class names) with symbolic names like 'VAR_1' and 'FUNC_3'. We trained our models on both datasets and compared the final results. Furthermore, the models were trained and evaluated in two scenarios:

- In the first scenario, the datasets were split into train and test sets with an 80:20 ratio.
- In the second scenario, the datasets were also split into train and test sets with the same ratio, but the test set was augmented by replacing all user-defined variables and method names with random strings (e.g., 'a', 'b', and 'c'). This aims to evaluate the robustness of the model against adversarial attacks, where the same vulnerable pattern may be disguised by features that are not relevant to the vulnerable pattern.

Model preparation: As demonstrated in the hyperparameter analysis results in Section 5.5.4, increasing the hidden size and decreasing the number of encoder layers of the model may improve the detection performance. Therefore, in this experiment, we employed a classifier containing three layers of transformer encoders with $h = 768$. To assess the effectiveness of models that were trained using attention mask M combined with code normalization steps, we used different feature extraction approaches, depending on whether the dataset was

normalized or unnormalized. For normalized datasets, we applied an attention mask to eliminate the attention signal generated by symbolic names, whereas a standard self-attention operator is utilized in each encoder block for unnormalized datasets.

Results: Table 5 provides a detailed description of the results obtained from our experiment. We observed a trade-off between precision and recall when training the model on normalized and unnormalized datasets in both scenarios.

In general, empirical results demonstrated that models trained on normalized datasets outperformed those trained on unnormalized datasets in terms of recall scores, which is a crucial metric that reflects the model's ability to accurately identify all vulnerable statements. Specifically, in the normal scenario, models trained on normalized code achieved an average recall score of 69.83% for CVEFixes and 72.32% for VUDENC, whereas models trained on the original code had slightly lower recall scores, with 62.15% for CVEFixes and 70.28% for VUDENC. The difference in the recall scores of the models trained on normalized and unnormalized data was even more significant in the second scenario, with models trained on unnormalized data achieving significantly lower recall scores (37.64% for CVEFixes and 42.63% for VUDENC), whereas the recall scores for models trained on normalized code were less affected (50.22% for CVEFixes and 60.52% for VUDENC). This is desirable, as the model needs to be able to withstand adversarial attacks and accurately identify patterns in vulnerable code statements.

In contrast, models trained on unnormalized data had better precision than those trained on normalized data, except for CWE types such as CWE-352 and CWE-601 in the CVEFixes dataset. Specifically, the average precision score for models trained on unnormalized code is 82.82% for CVEFixes and 63.30% for VUDENC, outperforming models trained on normalized code, which achieves a precision score of 72.67% for CVEFixes and 45.19% for VUDENC. The same pattern is observed in the second scenario, where the precision scores drop for both normalized and unnormalized models, but the precision scores of models trained on unnormalized data (73.71% for CVEFixes and 66.12% for VUDENC) are higher than those of models trained on normalized data (64.99% for CVEFixes and 37.62% for VUDENC). Another insight from Table 5 is that models trained on normalized datasets produce better AUROC scores for most cases. Since the AUROC score is a measure of how well a model can distinguish between vulnerable and nonvulnerable code, the higher the AUROC score, the model is more likely to recognize vulnerable patterns when presented with a vulnerable statement.

In summary, code normalization is a crucial step in improving the detection accuracy of our model. By removing the attention signal from symbolic names, code normalization allows the model to focus on the underlying code structure, resulting in identifying vulnerabilities more accurately. It also leads to a more aggressive detection behavior, thus significantly increasing the model's recall. In future work, we are going to focus on finding solutions to the challenge of maintaining a balance between recall and precision scores.

5.5.2. Experiment results for RQ2

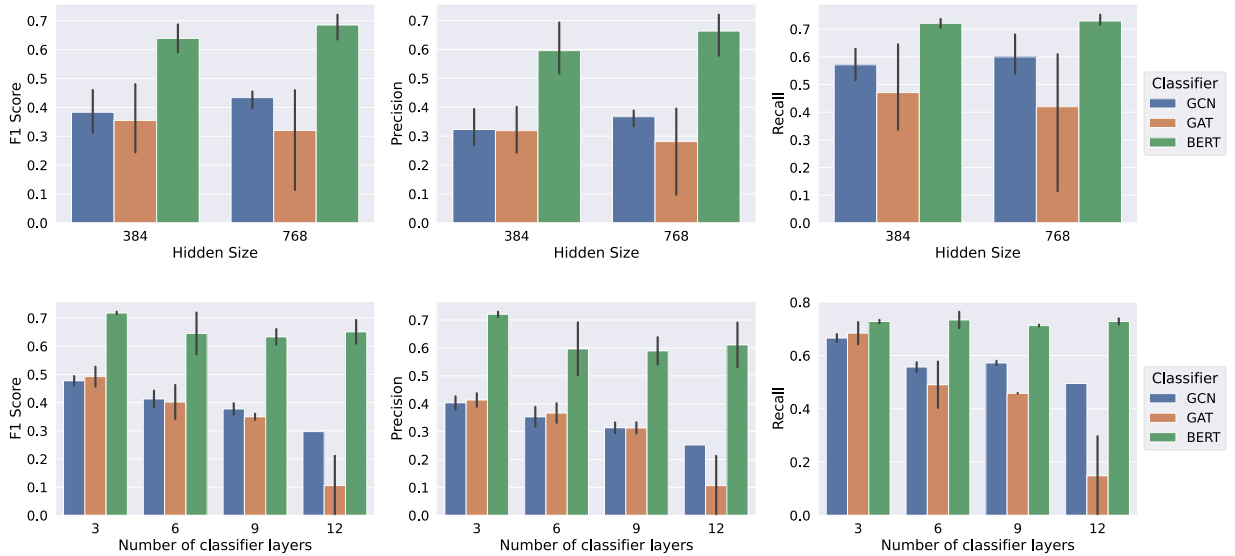
This experiment was conducted to answer the most important research question in this study which is whether DetectVul can replace GNN-based models to perform the statement-level SVD task.

Data preparation: To evaluate the effectiveness of DetectVul compared to GNN-based models (e.g., GCN and GAT), we use two datasets:

Table 5

Performance of DetectVul models in different scenarios on normalized and unnormalized versions of the two datasets - VUDENC and CVEFixes.

Evaluate scenarios	Dataset	Normalized	Metric	CWE-22	CWE-77	CWE-79	CWE-89	CWE-352	CWE-601	CWE-94	Average
Normal	CVEFixes	×	F1	51.61	n/a	53.66	n/a	80.95	72.09	90.41	69.74
			Prec	61.54	n/a	91.67	n/a	85.00	81.58	94.29	82.82
			Rec	44.44	n/a	37.93	n/a	77.27	64.58	86.84	62.15
			AUROC	63.74	n/a	77.71	n/a	98.69	90.79	95.65	85.30
	CVEFixes	✓	F1	42.30	n/a	54.55	n/a	93.02	75.48	86.19	70.30
			Prec	36.36	n/a	57.69	n/a	95.24	88.57	82.50	72.67
			Rec	50.56	n/a	51.72	n/a	90.91	65.76	90.23	69.83
			AUROC	80.12	n/a	77.42	n/a	97.81	94.43	99.64	89.88
	VUDENC	×	F1	63.42	66.67	42.90	64.88	72.38	73.25	78.79	66.04
			Prec	52.82	73.50	40.09	67.80	66.20	65.44	77.38	63.30
			Rec	79.37	60.99	46.15	62.20	79.83	83.18	80.25	70.28
			AUROC	92.38	92.77	84.72	86.43	93.51	96.27	89.94	90.86
	VUDENC	✓	F1	50.41	64.23	39.75	51.37	60.78	48.75	66.33	54.52
			Prec	36.60	66.17	33.33	42.41	47.21	34.72	55.93	45.19
			Rec	80.95	62.41	49.22	65.14	85.29	81.78	81.48	72.32
			AUROC	91.59	88.30	79.64	88.68	95.76	97.35	94.13	91.77
Augmented	CVEFixes	×	F1	23.08	n/a	27.03	n/a	58.82	69.23	67.80	49.20
			Prec	37.50	n/a	62.50	n/a	83.33	90.00	95.24	73.71
			Rec	16.67	n/a	17.24	n/a	45.45	56.25	52.63	37.64
			AUROC	64.36	n/a	68.09	n/a	99.04	90.50	97.53	82.70
	CVEFixes	✓	F1	30.30	n/a	24.00	n/a	74.29	73.81	77.92	56.06
			Prec	33.33	n/a	28.57	n/a	100.00	86.11	76.92	64.99
			Rec	27.78	n/a	20.69	n/a	59.09	64.58	78.95	50.22
			AUROC	63.89	n/a	74.46	n/a	98.62	94.34	99.04	86.07
	VUDENC	×	F1	48.48	48.11	45.45	45.45	71.16	48.33	50.82	51.11
			Prec	66.67	71.83	55.56	62.30	71.31	59.59	75.61	66.12
			Rec	38.10	36.17	38.46	35.78	71.01	40.65	38.27	42.63
			AUROC	90.76	82.94	76.28	78.89	94.54	91.55	85.77	85.81
	VUDENC	✓	F1	53.41	45.85	29.41	43.97	50.97	42.58	50.22	45.19
			Prec	41.59	51.79	23.81	38.54	36.82	31.35	39.44	37.62
			Rec	74.60	41.13	38.46	51.19	82.77	66.36	69.14	60.52
			AUROC	91.51	89.24	78.43	88.69	96.53	97.02	91.78	90.45

**Fig. 8.** We conducted experiments on various classifier architectures, using pretrained feature extractors explained in Section 5.4. The results were plotted as bar graphs, with the DetectVul model, which employed BERT as its classifier, represented by the green bar and the other bars indicating GNN-based architectures including GCN and GAT.

the dataset from RQ1 (the combination of VUDENC and CVEFixes datasets) and build a new CFG dataset. The CFG dataset consists of graphs that represent the control flow or the path of execution of a program, which is an essential tool for analyzing and understanding the program's behavior. It is also frequently used in previous studies [9,10,22,23,32]. For a fair comparison, we reused feature extractor models of DetectVul listed in Table 4 and replaced the classifier by GCN and

GAT. The same data preprocessing and training pipeline is also applied to extract statements and form function gadgets.

GNN-based models preparation: In the training process, we treat each CFG as a function gadget d_i where each node is a statement s_{ij} that has its corresponding label y_{ij} . All statements are fed to one of the feature extractors shown in Table 4 to generate contextualized dynamic embeddings. Then, we pass the generated embeddings to a GNN-based

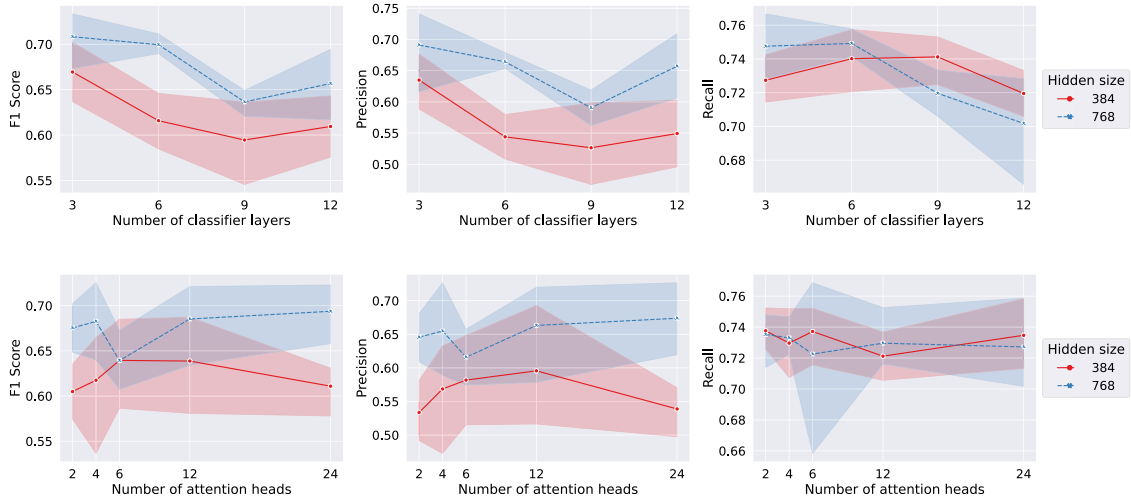


Fig. 9. Plots that are created to illustrate how DetectVul performs on three metrics - F1, precision, and recall scores - with different parameters.

Table 6

Performance of DetectVul and other GNN-based models.

FE	Classifier	F1	Precision	Recall
MiniLM	GCN	38.32 $\pm$ 8.24	32.98 $\pm$ 7.41	57.13 $\pm$ 6.36
	GAT	35.49 $\pm$ 13.04	31.88 $\pm$ 9.33	47.07 $\pm$ 18.25
	BERT	62.23 $\pm$ 5.21	56.37 $\pm$ 7.32	72.69 $\pm$ 2.05
mpnet	GCN	45.18 $\pm$ 1.28	38.42 $\pm$ 0.62	60.98 $\pm$ 10.03
	GAT	32.01 $\pm$ 21.85	28.11 $\pm$ 18.97	41.98 $\pm$ 28.99
	BERT	67.53 $\pm$ 4.38	65.07 $\pm$ 6.35	72.95 $\pm$ 3.19

model to produce final predictions. The architectures used for baseline GNNs are GCN [30] and GAT [31]. These models are trained under the same process of DetectVul. To fully compare the performance of our model and GNN-based approaches, we conducted several experiments on each architecture using various hyperparameters.

Results: As shown in Fig. 8, the top three plots indicate that increasing the hidden size has a positive impact on the performance of the models in terms of F1 score, precision, and recall. This finding is consistent across all classifier architectures. Specifically, our experiment showed that models with hidden size $h_{mpnet} = 784$ had an improvement of around 5.8%, 7.2%, and 3.9% in F1 score, precision, and recall, respectively over models with lower hidden size $h_{miniLM} = 384$.

The lower plots of Fig. 8 reveal another pattern. As the number of encoder layers increases, the performance of all GNN-based models tends to decrease across all metrics. This is commonly referred to as the over-smoothing [33] and over-quashing [34] problems, which can arise in the message-passing mechanism of GNN architectures. In essence, over-smoothing occurs when the GNN model aggregates excessive information from neighboring nodes, causing the feature vector of each node to lose its unique characteristics and become overly similar to those of its neighbors. As a result, the model may struggle to distinguish between different nodes and make accurate predictions. On the other hand, over-quashing occurs when the GNN model compresses the information obtained from neighboring nodes, leading to the loss of critical features for making accurate predictions. Besides these issues in GNN-based models, we also observed a decline in DetectVul's precision metric as the number of encoder layers increased. However, the recall metric, which measures the model's ability to identify all vulnerable statements correctly, is stable. This suggests that DetectVul's ability to capture and retain important information from the input was not compromised as the number of encoder layers increased.

Table 7

Effect of learnable statement type embeddings.

Dataset	Use STE	F1	Precision	Recall
CVEFixes	×	62.23 $\pm$ 4.95	56.11 $\pm$ 8.45	73.02 $\pm$ 3.34
	✓	63.07 $\pm$ 9.62	58.09 $\pm$ 14.30	72.01 $\pm$ 1.06
Vudenc	×	55.19 $\pm$ 3.34	45.94 $\pm$ 5.63	73.82 $\pm$ 4.92
	✓	62.99 $\pm$ 4.85	50.29 $\pm$ 5.22	74.04 $\pm$ 2.56

As shown in Table 6, the average F1, precision, and recall scores of DetectVul are nearly double those of models that used GCN and GAT as classifiers. The explanation for this improvement was discussed in Section 4.4, where the classifier model of DetectVul created multiple attention patterns at each attention head of each encoder layer, resulting in contextualized vectors that represented combinations of multiple weighted graph structures. For example, using a DetectVul classifier with 12 attention heads and a hidden size of 768 is equivalent to incorporating 12 distinct graph structures at each encoder layer, with each node having an embedding vector of size 64.

5.5.3. Experiment results for RQ3

To enhance the performance of our model, we observed that vulnerabilities often appear in some specific types of statements, such as conditions, expressions and assignments. As a hypothesis, we propose that supplying additional information about the statement type for each feature vector before feeding into the classifier could lead to improvements. This section is provided to discuss the effect of the statement type embeddings on the performance of DetectVul.

Model and data preparation: In this experiment the datasets from Section 5.5.1 are reused, however this time we remove the statement type embeddings and train all models from scratch.

Results: In Table 7, we can see that, while models lacking statement type embedding have slightly improved recall scores, their precision scores experience a substantial decline on the CVEFixes datasets, consequently resulting in lower F1 scores. A similar phenomenon is also repeated in the Vudenc dataset but is less significant due to its large amount of data. These outcomes confirm that the inclusion of statement-type embeddings aids the model in more precisely identifying vulnerable statements. For more details on DetectVul's performance across each statement type, please refer to Appendix D.

5.5.4. Analysis of parameters

This section includes information about how different hyperparameter settings can influence the performance of DetectVul. Specifically, we trained different versions of DetectVul by adjusting the hidden size, number of encoder layers, and number of attention heads in its classifier.

Model and data preparation: For simplicity, the VUDENC and CVEFixes datasets are merged for training and evaluation. This is also necessary, as the CVEFixes dataset has a limited number of samples related to classes CWE-89 and CWE-77. Each model was trained for 100 epochs using the training strategy discussed in Section 4.3.3.

Results: From Fig. 9, we can see that models with larger hidden sizes always have better performance in all metrics. Specifically, when comparing models with smaller hidden size $h_{MiniLM} = 384$ to those with larger hidden size $h_{mpnet} = 768$, the smaller models achieved competitive recall scores ranging from 69.96% to 77.85%. However, given the same set of other hyperparameters (number of classifier layers and attention heads), the larger models consistently achieved better precision scores ranging from 54.05% to 77.10% compared to smaller models, which ranged from 42.50% to 71.10%. Consequently, the larger models also achieved better F1 scores ranging from 59.64% to 74.47%, whereas smaller models with hidden size $h = 384$ only achieved F1 scores ranging from 49.70% to 71.33%. These results align with previous studies such as [20,61] which suggest that larger hidden sizes enable models to capture more complex features, thus improving their ability to generalize to new data.

Another interesting point is that the number of encoder layers does not significantly impact the performance of DetectVul. In fact, models with fewer classifier layers tended to perform better. One possible explanation for this observation is the risk of overfitting. When the number of encoder layers is high, the model may become too specialized for the training data, resulting in poor generalization performance on new data. Therefore, as discussed in Section 4.4, using fewer classifier layers might better balance the complexity and generalize ability of the model.

6. Discussion

This section provides key theoretical contributions that position our research as well as the limitations of DetectVul. We also discuss potential approaches for future work.

6.1. Key findings and practical implications

We summarize the key findings and practical implications of our proposal in three main points:

- Firstly, to the best of our knowledge, we are the first to propose a deep-learning approach for predicting statement-level vulnerabilities in a scripting language, specifically Python. Unlike prior GNN-based methods tailored for C/C++, which needed multiple tools for graph construction, our approach only relies on Abstract Syntax Trees (AST) for data processing which makes it very easy to extend our work to other programming languages.
- Secondly, the empirical results show that our DetectVul not only simplifies the data preprocessing steps but also achieves better performance over other GNN-based models that are trained on CFG. This is explained in Section 4.4, in which we can see that by using multi-head attention the classifier model of DetectVul is performing ensemble learning with each head playing as one separate GNN model. This approach enables the model to freely explore vulnerable patterns among statements in Python functions without being confined to predefined graph structures.

Table B.8

Information about the number of vulnerable and non-vulnerable samples in the processed Devign dataset.

Type	No of function gadgets	No of statements
Non vulnerable	14 858	1030313
Vulnerable	12 460	31 857

Table C.9

Performance of VUDENC and other RNN-based models.

FE	Classifier	F1	Precision	Recall
Word2Vec	LSTM [8]	12.80 ± 2.78	9.72 ± 3.28	18.66 ± 2.67
MiniLM	LSTM	34.33 ± 2.78	29.72 ± 2.43	40.66 ± 3.24
mpnet	LSTM	38.91 ± 4.06	32.52 ± 3.72	48.43 ± 4.47

- Thirdly, in Section 5.5.2, factors such as variable and function names can also introduce noises into the training data and harm the performance of the models. By removing the attention signal from normalized symbolic names, the model can focus more on the underlying code pattern, such as the API/function call and the logic of each code expression. A better visualization of the effect of the code normalization is provided in Appendix A. In addition, the empirical results in Sections 5.5.1 and 5.5.4 showed that while increasing the hidden size of the model often leads to better performance, increasing the number of layers in the classifier model generally causes the model to overfit. This is also explained in Section 4.4 where the actual learning is performed in just the first three self-attention layers.

6.2. Limitations and future works

Despite having several advantages in detecting vulnerable statements over other GNN-based methods, certain limitations relating to the data preprocessing pipeline and model architecture still exist and need further enhancement in the future.

Data preprocessing pipeline: Firstly, the data preprocessing pipeline is heavily reliant upon the AST module for extracting user-defined names from code components. Although AST is used in almost every popular programming language, it might operate differently in each version. For example, the AST module for Python 3 cannot parse source code written in earlier versions of Python. This constraint imposes limitations on the scalability of our approach. Secondly, due to parsing source code into function-level artifacts known as “gadgets”, the context in which these gadgets operate is restricted to the scope of a single function. To mitigate the issues relating to the AST module, researchers should explore alternative data preprocessing methods. Ideally, the data preprocessing pipeline should not rely on any built-in library from programming languages. One potential approach is to use text segmentation or part-of-speech tagging to preprocess code data.

Model complexity: While DetectVul benefits from enabling parallelism for faster training and inference and eliminating the expensive graph construction step, facilitating on-the-fly inference, another drawback of DetectVul is its high space complexity. Since the space complexity of the self-attention operator in the classifier model is $O(n^2)$, where n is the number of statements, when n is large, DetectVul requires high computational resources for training and inference. For example, given functions, each with approximately $n = 128$ statements, a machine with 12 GB of GPU RAM can typically process a maximum of 32 function gadgets simultaneously, which might take DetectVul a long time to scan through repositories with a large codebase. To further improve the performance and efficiency of the DetectVul model. Future approaches can utilize ideas from BERT variants (e.g., ALBERT [49], DistilBERT [48]) or other transformer architectures (e.g., XLNet [51], Reformer [62], Longformer [50], Perceiver [63]) that are designed to optimize the time and space complexity efficiency. Moreover, another

Table D.10

Performance of DetectVul on each statement type in CVEFixes (left) and Vudenc (right)

Dataset	Type	Number of statements	F1	MCC	AUROC
CVEFixes	Condition	1660	83.99	0.8238	0.9819
	Return	1110	87.20	0.8767	0.9441
	Assign	3524	72.42	0.6928	0.8530
	Expression	1697	73.07	0.6819	0.7694
	For/While	251	49.96	0.8156	0.6606
	Assert	79	41.33	0.6986	0.5480
	Import From	365	33.12	0.4163	0.4571
	Augmented assign	44	16.66	0.0000	0.0000
	Import	115	16.66	0.0000	0.0000
	Function declaration	67	16.62	0.0000	0.0000
	Docstring	393	16.64	0.0000	0.0000
Vudenc	Expression	4983	69.42	0.5852	0.9082
	Assign	9001	62.79	0.5940	0.9031
	Return	2629	51.17	0.4768	0.8816
	Import From	1122	48.87	0.4057	0.8045
	For/while	756	47.52	0.5185	0.7717
	Condition	3613	43.85	0.4492	0.7655
	Augmented assign	170	45.43	0.6758	0.6158
	Import	674	29.17	0.3572	0.5503
	Assert	192	48.07	0.7405	0.4998
	Docstring	918	12.50	0.0000	0.0000
	Function declaration	2827	12.48	0.0000	0.0000

Table E.11

Glossary.

Acronyms	Full form
MLP	Multi-Layer Perceptrons
FE	Feature Extractors
BERT	Bidirectional Encoder Representations from Transformers [20]
STE	Statement Type Embeddings
CLSPooling	Pooling layer using special [CLS] token
PE	Position Embedding
WCE	Weighted Cross Entropy loss
GAT	Graph Attention networks [31]
GCN	Graph Convolution Networks [30]
CFG	Control Flow Graph
DPG	Data Dependency Graph
CPG	Code Property Graph
PDG	Program Dependency Graph
CG	Call Graph
GNN	Graph Neuron Network
SVD	Source code vulnerability Detection
CWE	Common Weakness Enumerations
CVE	Common Vulnerabilities and Exposures
NVD	National Vulnerability Database

promising option is to adopt the Space-State models [64], which is renowned for efficiently handling long sequences with a low memory footprint and without the fixed context window limitations of Transformers, enabling more scalable and faster processing of extensive input data.

7. Conclusion

In this paper, we introduced DetectVul, a BERT-based architecture that leverages the self-attention mechanism to detect vulnerable statements for Python. Unlike existing methods that rely on predefined graph structures, DetectVul is flexible and able to capture contextual interactions between statements, which makes it easy to apply our approach to other programming languages. To evaluate the effectiveness of DetectVul, we created a new dataset by extracting and normalizing statements from public datasets originating from open-source Python projects on GitHub, resulting in a total of 211,317 statements grouped into 21571 function gadgets. Empirical results demonstrated that DetectVul outperformed GNN-based architectures (e.g., GAT and

GCN) that relied on CFGs and achieved best F1 score of 74.10%, precision of 77.10%, and recall of 77.85% for detecting vulnerable Python statements. Moreover, the data normalization and statement type embedding module also help enhance the model's ability to generalize to new, unseen data and increase its robustness against adversarial attacks. In the future, we will conduct more experiments to benchmark DetectVul's performance against GNNs trained using other graph structures like PDG or CG and performing SVD tasks in other programming languages.

CRedit authorship contribution statement

Hoai-Chau Tran: Writing – review & editing, Writing – original draft, Visualization, Software, Methodology, Formal analysis, Data curation, Conceptualization. **Anh-Duy Tran:** Writing – review & editing, Validation, Project administration, Formal analysis. **Kim-Hung Le:** Writing – review & editing, Project administration, Methodology, Funding acquisition, Formal analysis, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

I have share the link of my code and data in the article.

Acknowledgment

This research was supported by The VNUHCM-University of Information Technology's Scientific Research Support Fund.

Appendix A. Effect of code normalization

In section is denoted to provide insights about how the code normalization steps described in Section 4.2 can help the model better resist adversarial attacks. The models used in RQ2 are utilized to generate embeddings for visualization.

The visualization presented in Fig. A.10 reveals that both models attempted to differentiate between vulnerable and nonvulnerable statements by separating their embeddings into distinctive regions. The unnormalized code-trained model produced clearer and more isolated clusters of vulnerable embeddings, forming “islands” within the plot, whereas the normalized code-trained model pushed vulnerable embeddings toward the edges of the two-dimensional planes.

These findings explain why the model trained on unprocessed data achieved higher precision scores, as the isolated “islands” provided a clear boundary to distinguish between vulnerable and nonvulnerable statements. However, the model trained on unnormalized code tended to be overfitted by irrelevant features leading to several false negative cases being missed, which are represented by colored markers that are scattered all over the plot.

In contrast, the normalized-trained model was able to identify the underlying patterns and structure of the code, despite the presence of irrelevant features. By removing inconsistencies such as variable and method names, we can better enhance the model to map statements using the same API calls, and function calls into unified vectors that are closer to each other in the vector space. Furthermore, the vulnerable embeddings produced by the normalized model were more prone to being pushed toward the right of the plot. This also explains why the model trained on normalized code achieved higher recall and AUROC scores.

Table G.12

This is a utility function that generates an HTML tag for a user's Gravatar image, setting its size and alternative text based on the user's details, or returns an empty string if no Gravatar is found.

CWE-77: Command Injection

```

1  def FUNC_2(VAR1, VAR2, VAR3):
2  """docstring"""
3  VAR_5 = get_gravatar_url(VAR_1['request'], VAR_2, VAR_3)
4  if VAR_5:
5      return '' % (
6          VAR_5, VAR_3, VAR_3, VAR_2.get_full_name() or VAR_2.username)
7  return ''

```

The use of the string formatting operation (%) line 5, 6 to directly insert VAR_5, VAR_3, and VAR_2.get_full_name() or VAR_2.username into the HTML tag can be dangerous if any of these variables contain special characters or malicious payloads. Although this example is within an HTML context rather than a direct shell command, an attacker could craft input that breaks out of the intended context (e.g., inserting a <script> tag or other HTML/JavaScript injection).

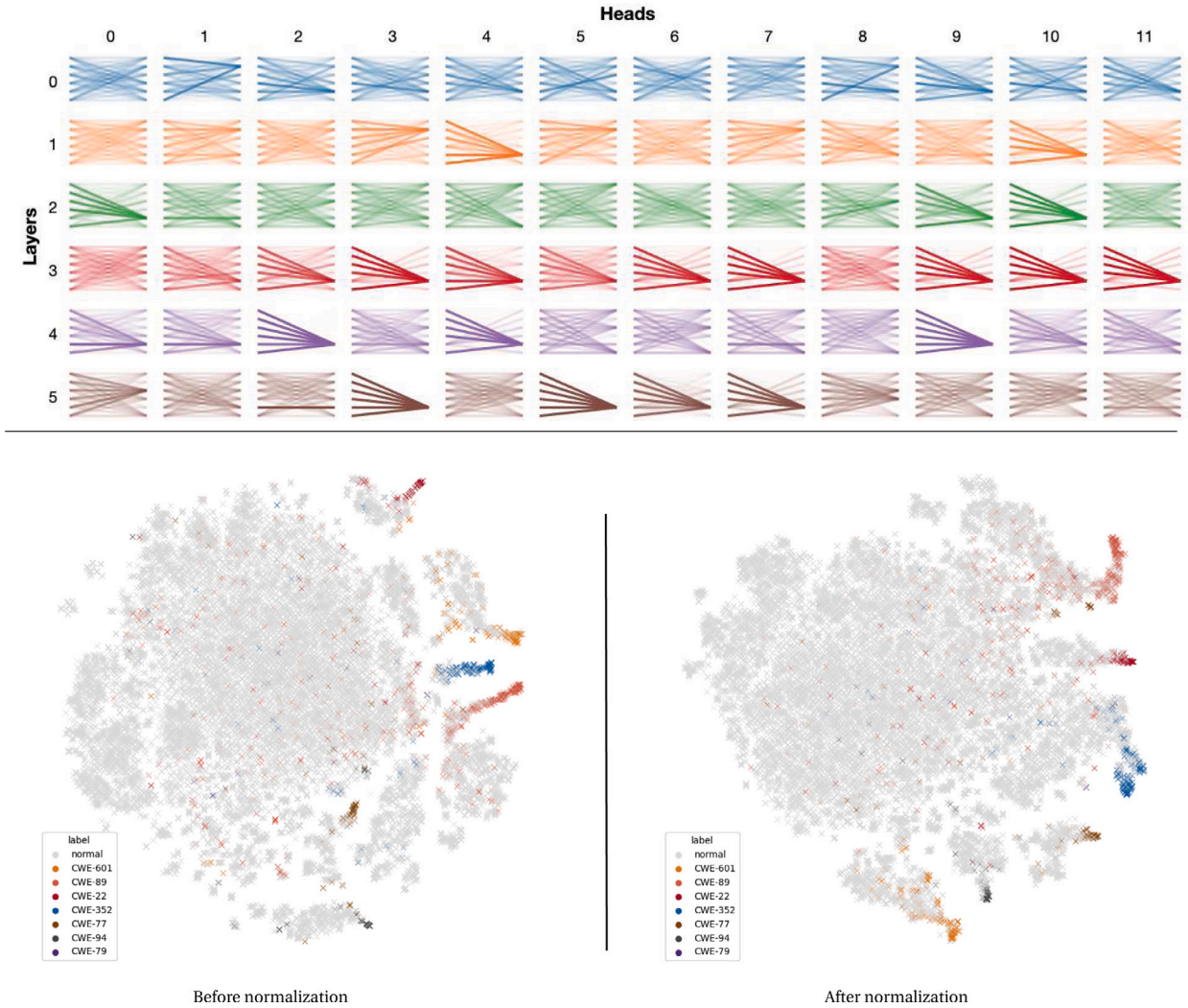


Fig. A.10. The t-SNE algorithm is utilized to visualize the 2D scatter plots of the final embedding vectors of all statements (s_{ij}) in the augmented test set. These embedding vectors are generated by the last layer of the encoder model, which has a hidden size of 786 before being projected and classified. Each statement is assigned an 'x' marker with one specific color which represents the ground truth label of the current statement.

Appendix B. New C/C++ dataset for statement-level SVD task

As C/C++ is widely used in previous studies for the source code vulnerability detection task, we also include this section to introduce our new dataset which originated from the famous Devign dataset [10]. Table B.8 shows a brief statistic of the new dataset for C/C++.

As the Devign dataset is collected for function-level memory-related vulnerability detection, we had to use our data preprocessing pipeline as proposed in Section 4.2 to extract, label and normalize statements. The module used to extract and standardize statements was PyClang. The final processed Devign dataset can be found on our Hugging-face hub: https://huggingface.co/datasets/EddieChen372/devign_with_

Table G.13

The function iterates over a list of file paths, attempts to open each file from a cloud storage system, logs any “file not found” errors, and yields the opened files.

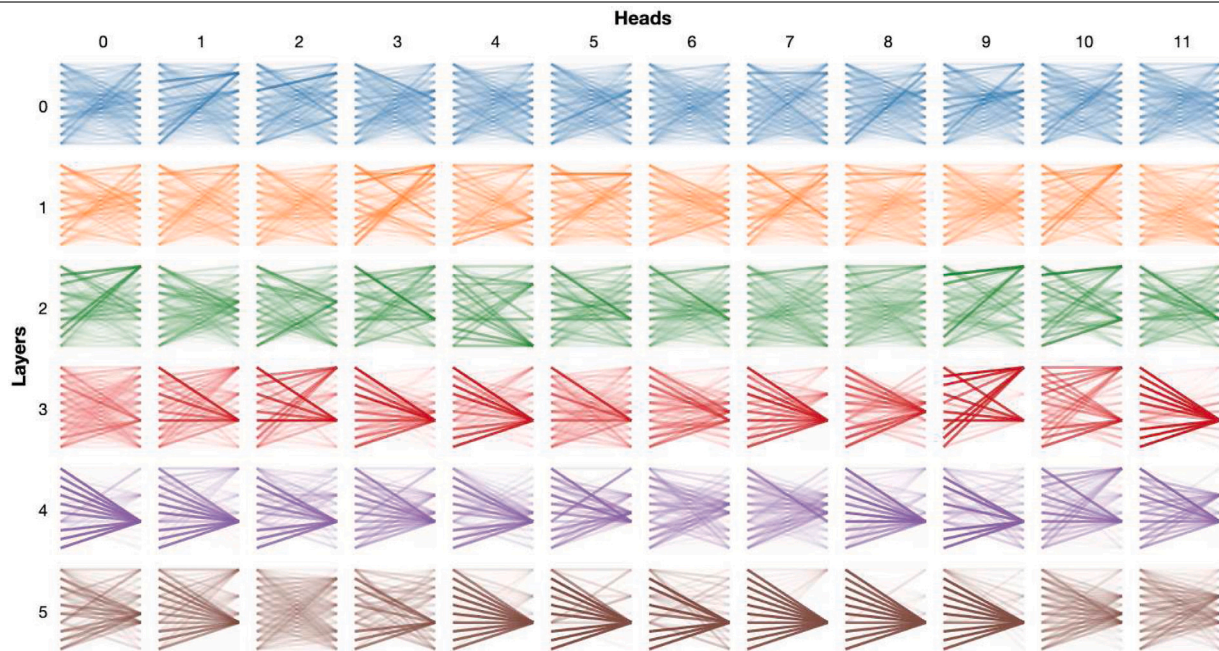
CWE-22: Path Traversal

```

1 def FUNC_1(VAR_3)
2     """docstring"""
3     for VAR_5 in VAR_3:
4         VAR_5 = pathlib.PurePosixPath(VAR_5)
5         VAR_6 = storage.aws_location
6         VAR_6 = storage.location
7         VAR_7 = storage.open(str(VAR_5.relative_to(VAR_6)))
8         VAR_0.exception('File not found: %s', VAR_5)
9         VAR_7.name = VAR_5.name
10    yield VAR_7

```

The line 7, `VAR_7 = storage.open(str(VAR_5.relative_to(VAR_6)))` attempts to open a file based on a path relative to `VAR_6`. If `VAR_5` is crafted to contain a path that traverses directories (e.g., `../../etc/passwd`), this could result in the application opening or processing unintended files, leading to unauthorized access to sensitive files.



```

103     if (!unit->data[len - 1]) {
104         int z;
105         for (
106             z = 0;
107             z < len && !unit->data[len - z - 1]
108             ; z++)
109             ;
110         av_log(ctx->log_ctx, AV_LOG_DEBUG,
111             "Deleted %d trailing zeroes"
112             "from slice data.\n", z);
113         len -= z;
114     }
115     slice->data_size = len - pos / 8;
116     slice->data = av_malloc(slice->data_size);
117     if (!slice->data) {
118         av_free(slice);
119         return AVERROR(ENOMEM);
120     }
121     memcpy(slice->data,
122         unit->data + pos / 8,
123         slice->data_size);
124     slice->data_bit_start = pos % 8;
125     unit->content = slice;
126 }
127 break;

```

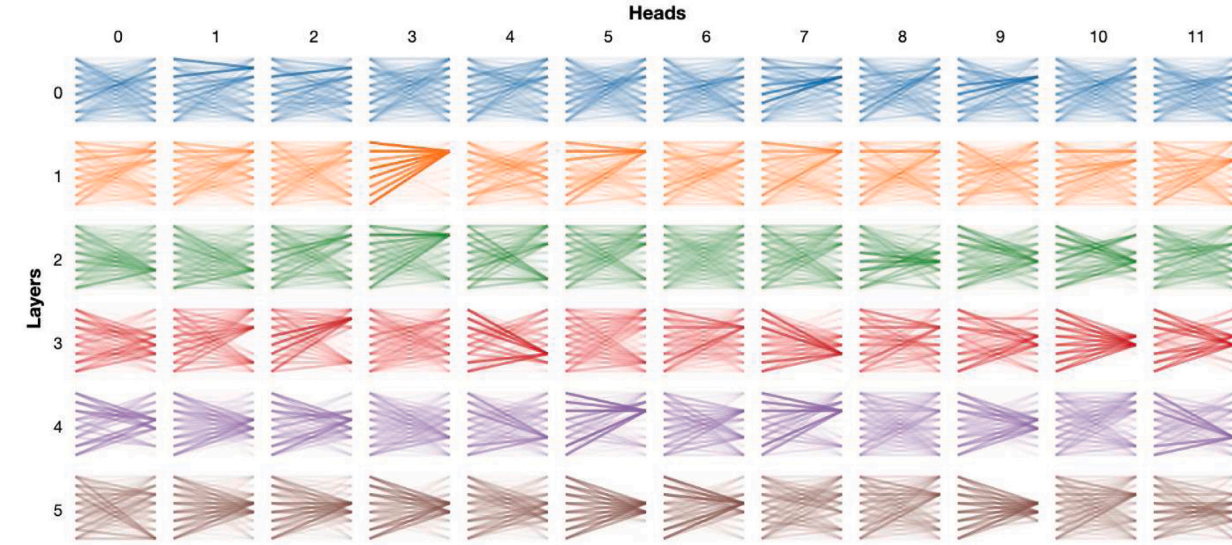
Fig. B.11. A use case where our model detected a memory-related vulnerability in C/C++ source code.

Table G.14

The function retrieves a course, student, and unit based on provided identifiers, sets a due date extension for the student in the specified unit, and returns a JSON response confirming the successful change.

CWE-79: Cross-Site Scripting	
1	<code>def FUN_28(VAR_10, VAR_9)</code>
2	<code> """docstring"""</code>
3	<code> VAR_65 = get_course_by_id(SlashSeparatedCourseKey.from_deprecated_string(VAR_10))</code>
4	<code> VAR_40 = require_student_from_identifier(VAR_9.GET.get('student'))</code>
5	<code> VAR_36 = find_unit(VAR_65, VAR_9.GET.get('url'))</code>
6	<code> VAR_144 = parse_datetime(VAR_9.GET.get('due_datetime'))</code>
7	<code> set_due_date_extension(VAR_65, VAR_36, VAR_40, VAR_144)</code>
8	<code> return JsonResponse(_('Successfully changed due date for student {0} for {1} to {2}').format(</code> <code> VAR_40.profile.name, FUNC_59(VAR_36), VAR_144.strftime('%Y-%m-%d %H:%M')))</code>

In line 6,7, the code directly incorporates user input from VAR_9.GET into the response message without proper sanitization or escaping. Specifically: (1) The student's identifier (VAR_40) is retrieved from VAR_9.GET.get('student') and then return as json response. (2) The due_datetime (VAR_144) and url parameters are also derived from user input.



[norm_vul_lines](#). Future researchers can use this dataset to benchmark their work for statement-level SVD for C/C++.

In [Fig. B.11](#) After training on the new Devign dataset, our model was able to detect a statement that is vulnerable to CWE-119 (buffer overflow) or CWE-787 (out-of-bounds read or write). Specifically, the statement in line 116, which is marked as vulnerable, allocates a block of memory of size 'slice->data_size' using the 'av_malloc' function. If 'slice->data_size' is a very large value (for example, if an attacker can control the input to the function), the 'av_malloc' function may fail to allocate enough memory and return a 'NULL' pointer. If the program does not handle this error condition properly, it could lead to a buffer overflow or other memory-related vulnerabilities.

Appendix C. Performance of VUDENC and RNN-based models

This experiment was conducted to benchmark the performance of RNN-based model on our new dataset.

Data and models preparation: In this experiment, we used the same dataset and the RNN-based models reused LLM feature extractors as the models used in [Section 5.5.2](#). Multiple layers of LSTM were utilized as the final classifier. After feeding all statements into the feature extractor, the feature vectors are then fed sequentially into the classifier to produce the final result.

As expected, the results from [Table C.9](#) is far worse than those of the GNN-based approach and DetectVul presented in [Table 5](#). This is reasonable since the model can only see one statement at each step, and

the context signals about previous statements are compressed into one single vector which gradually loses information after each step. Note that the result of the Vudenc model in the original paper is high (up to 90%) because the data is severely leaked between the train and test set, leading to over-optimistic performance. In reality, we observed that Vudenc performed very badly on statement-level and even slice-level SVD with a large window size of 200 (*i.e.* 200 tokens for each slice).

Appendix D. Performance of DetectVul on each statement type

To assess DetectVul's effectiveness on each statement type, we analyzed whether models accurately classified the same types of statements as vulnerable in both datasets. As ground truth labels may be imperfect, leading to false positives, another metric called Matthews Correlation Coefficient (MCC) was used in conjunction with F1.

$$MCC = \frac{TP \times TN - FP \times FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}} \quad (D.1)$$

$$MCC_{macro} = \frac{\sum_{i=0}^{c-1} MCC_i}{c} \quad (D.2)$$

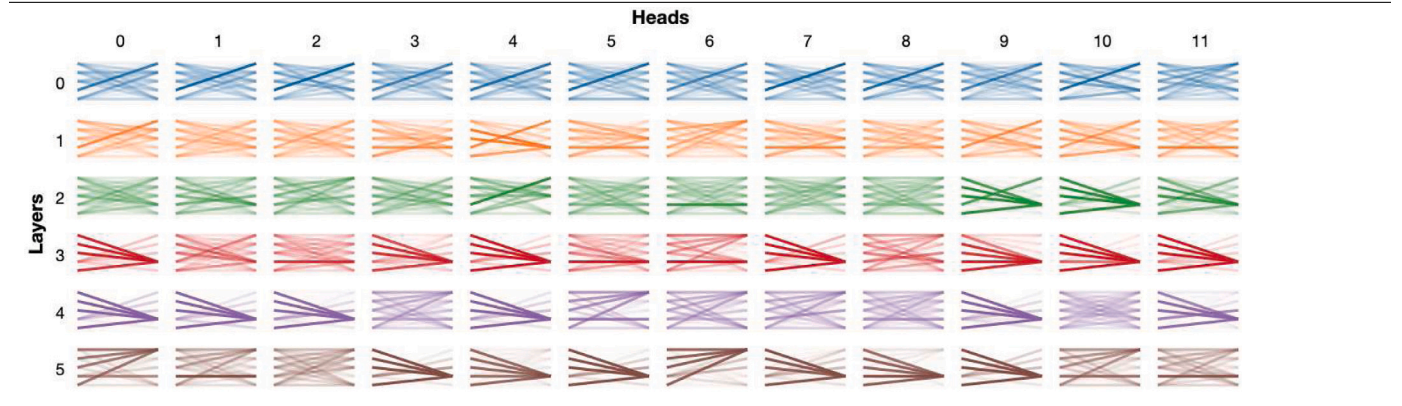
In [\(D.1\)](#), we can see that, unlike F1, MCC is not dependent on true negative (TN) predictions. The MCC formula produces an output value between -1 and 1, and [\(D.2\)](#) is the macro average formula for the MCC score of all classes $i \in \{0, 1, 2, \dots, c-1\}$. When the MCC has a value of 1, it indicates that the model's predictions align perfectly with the provided ground truth labels, otherwise, a -1 score indicates a completely different prediction from ground truth labels.

Table G.15

The function creates a command-line argument parser, constructs arguments to load checkpoint of a deep learning model, optionally including a `--use_tftrt` flag, and then executes the saved model using the parsed arguments.

CWE-352: Cross-Site Request Forgery	
1	<code>def FUNC_5(VAR_0, VAR_5)</code>
2	<code>self.parser = saved_model_cli.create_parser()</code>
3	<code>VAR_10 = test.test_src_dir_path(VAR_0)</code>
4	<code>VAR_11 = self.parser.parse_args(['run', '--dir', VAR_10, '--tag_set', 'serve', '--signature_def', 'INVALID_SIGNATURE', '--input_exprs', 'x2=np.ones((3,1))'] + (['--use_tftrt'] if VAR_5 else []))</code>
5	<code>saved_model_cli.run(VAR_11)</code>

The code uses `saved_model_cli.run(VAR_11)` in line 5, which appears to execute a command or process with arguments obtained from user input in line 4. The arguments for this command, including `VAR_10` and `VAR_11`, come from `self.parser.parse_args()` which parses command-line arguments. If `VAR_11` includes user-controlled inputs, such as paths or parameters, an attacker could craft input that might be executed in a way that affects the system or application.



Results: Table D.10 presents the performance of DetectVul for each statement type on the evaluation set of the CVEFixes and VUDENC datasets. All statement types in each dataset are arranged by AUROC score in descending order.

In the CVEFixes dataset the model performs exceptionally in detecting vulnerabilities in “Return”, “Condition”, “For/While”, “Assign” and “Expression” statements, as it achieved the highest scores for all metrics. However, the model seemed to struggle in identifying vulnerabilities in “Augmented assignment”, “Import”, “Function declaration”, and “Docstring” statements, with scores being very low or zero. This could be attributed to the limited number of samples available for these types of statements or the fact that they are not actually vulnerable, which is a positive signal indicating that the model can distinguish between statements that have the potential to contain vulnerable patterns and the statements that are labeled as vulnerable just because they are deleted in the commits that patch the vulnerable code. We can also see the same pattern observed in the Vudenc dataset, where the model performed best in detecting vulnerabilities in “Condition”, “Assign”, “Expression”, “Return”, and “For/While” statements with high AUROC scores.

Appendix E. CWE description

This section is devoted to briefly discussing each CWE that was used in this paper:

- **CWE-22: Improper Limitation of a Pathname to a restricted directory (Path Traversal)** - A vulnerability that occurs When a web application fails to validate user input, allowing users to access files or directories beyond the intended location. This can result in the unauthorized access of critical system files or the exposure of sensitive information.
- **CWE-79: Improper Neutralization of Input During Web Page Generation (Cross-site scripting)** - A vulnerability in web applications

that occurs when untrusted user input is not properly sanitized or escaped, allowing an attacker to inject malicious code into the web page and execute actions on behalf of the victim.

- **CWE-77: Improper Neutralization of Special Elements used in a Command (Command Injection)** - A vulnerability occurs when an interpreter or command shell receives untrusted user input as a component of a command, allowing attackers to execute arbitrary code or commands on the system.
- **CWE-89: Improper Neutralization of Special Elements used in an SQL Command (SQL Injection)** - A vulnerability that occurs when user input is not properly sanitized before being used in an SQL query, allowing an attacker to inject malicious SQL code to access and modify sensitive data.
- **CWE-94: Improper Control of Generation of Code (Code Injection)** - A vulnerability that occurs when untrusted user input generates executable code, allowing attackers to inject and execute arbitrary code on the system.
- **CWE-352: Cross-Site Request Forgery (CSRF)** - A vulnerability that occurs when an attacker manipulates a victim into unknowingly executing an action on a website or application. This occurs by exploiting the victim’s authenticated session on the targeted site, leading them to unintentionally perform actions they did not intend to.
- **CWE-601: URL Redirection to Untrusted Site (Open Redirect)** - A vulnerability that occurs when a web application redirects a user to an untrusted or malicious site, which an attacker can exploit to steal sensitive information or perform other malicious actions.

Appendix F. Glossary

To enhance readability and ensure clarity throughout this paper, we have provided a glossary table, as shown in Table E.11, helping readers unfamiliar with certain terms to easily find their full forms and understand their meanings in the context of this research.

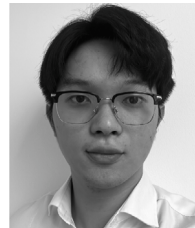
Appendix G. More visualization

To further validate the explanation provided in Section 4.4, this section presents four additional visualizations. Each visualization includes a description of the function's behavior, the type of vulnerability, the cause of the vulnerability, and a graphical depiction of how attention heads focus on each statement. In general, a consistent pattern emerges across the visualizations, where the classifier model forms distinct attention patterns in each head, and in the final layers, most attention heads can identify the vulnerable statements accurately (see Tables G.12–G.15).

References

- [1] Dean, What is code vulnerability? Codegrip (2020) URL <https://www.codegrip.tech/productivity/what-is-code-vulnerability/>.
- [2] Bushra Aloraini, Meiyappan Nagappan, Daniel M. German, Shinpei Hayashi, Yoshiaki Higo, An empirical study of security warnings from static application security testing tools, *J. Syst. Softw.* 158 (2019) 110427, <http://dx.doi.org/10.1016/j.jss.2019.110427>, URL <https://www.sciencedirect.com/science/article/pii/S0164121219302018>.
- [3] C. Lattner, V. Adve, LLVM: a compilation framework for lifelong program analysis & transformation, in: *International Symposium on Code Generation and Optimization*, 2004, CGO 2004, 2004, pp. 75–86, <http://dx.doi.org/10.1109/CGO.2004.1281665>.
- [4] Cristian Cadar, Daniel Dunbar, Dawson Engler, KLEE: Unassisted and automatic generation of high-coverage tests for complex systems programs, in: *Proceedings of the 8th USENIX Conference on Operating Systems Design and Implementation*, OSDI '08, USENIX Association, USA, 2008, pp. 209–224.
- [5] Mark Weiser, Program slicing, *IEEE Trans. Softw. Eng.* SE-10 (4) (1984) 352–357, <http://dx.doi.org/10.1109/TSE.1984.5010248>.
- [6] Control Flow (GNU Compiler Collection (GCC) Internals), URL <https://gcc.gnu.org/onlinedocs/gccint/Control-Flow.html>.
- [7] Joernio, GitHub - joernio/joern: Open-source code analysis platform for C/C++/Java/Binary/JavaScript/Python/Kotlin based on code property graphs. Discord <https://discord.gg/vv4MH284Hc> URL <https://github.com/joernio/joern>.
- [8] Laura Wartschinski, Yannic Noller, Thomas Vogel, Timo Kehler, Lars Grunske, VUDENC: Vulnerability detection with deep learning on a natural codebase for Python, *Inf. Softw. Technol.* 144 (2022) 106809, <http://dx.doi.org/10.1016/j.infsof.2021.106809>.
- [9] Saikat Chakraborty, Rahul Krishna, Yangruibo Ding, Baishakhi Ray, Deep learning based vulnerability detection: Are we there yet?, 2020, <http://dx.doi.org/10.48550/ARXIV.2009.07235>, URL <https://arxiv.org/abs/2009.07235>.
- [10] Yaqin Zhou, Shangqing Liu, Jingkai Siow, Xiaoning Du, Yang Liu, Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks, 2019, <http://dx.doi.org/10.48550/ARXIV.1909.03496>, URL <https://arxiv.org/abs/1909.03496>.
- [11] Yi Li, Shaohua Wang, Tien N. Nguyen, Vulnerability detection with fine-grained interpretations, in: *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, in: ESEC/FSE 2021, Association for Computing Machinery, New York, NY, USA, 2021, pp. 292–303, <http://dx.doi.org/10.1145/3468264.3468597>.
- [12] Sicong Cao, Xiaobing Sun, Lili Bo, Ying Wei, Bin Li, BGNN4vd: Constructing bidirectional graph neural-network for vulnerability detection, *Inf. Softw. Technol.* 136 (2021) 106576, <http://dx.doi.org/10.1016/j.infsof.2021.106576>.
- [13] Rebecca L. Russell, Louis Kim, Lei H. Hamilton, Tomo Lazovich, Jacob A. Harer, Onur Ozdemir, Paul M. Ellingwood, Marc W. McConley, Automated vulnerability detection in source code using deep representation learnings, 2018, <http://dx.doi.org/10.48550/ARXIV.1807.04320>, URL <https://arxiv.org/abs/1807.04320>.
- [14] Deqing Zou, Sujuan Wang, Shouhuai Xu, Zhen Li, Hai Jin, μ VulDeePecker: A deep learning-based system for multiclass vulnerability detection, *IEEE Trans. Depend. Secure Comput.* (2019) 1, <http://dx.doi.org/10.1109/tdsc.2019.2942930>.
- [15] Z. Li, D. Zou, S. Xu, H. Jin, Y. Zhu, Z. Chen, SySeVR: A framework for using deep learning to detect software vulnerabilities, *IEEE Trans. Dependable Secure Comput.* 19 (04) (2022) 2244–2258, <http://dx.doi.org/10.1109/TDSC.2021.3051525>.
- [16] Zhen Li, Deqing Zou, Shouhuai Xu, Zhaoxuan Chen, Yawei Zhu, Hai Jin, VulDeeLocator: A deep learning-based fine-grained vulnerability detector, *IEEE Trans. Depend. Secure Comput.* 19 (4) (2022) 2821–2837, <http://dx.doi.org/10.1109/tdsc.2021.3076142>.
- [17] Zhen Li, Deqing Zou, Shouhuai Xu, Xinyu Ou, Hai Jin, Sujuan Wang, Zhijun Deng, Yuyi Zhong, VulDeePecker: A deep learning-based system for vulnerability detection, in: *Proceedings 2018 Network and Distributed System Security Symposium*, Internet Society, 2018, <http://dx.doi.org/10.14722/ndss.2018.23158>.
- [18] Haşim Sak, Andrew Senior, Françoise Beaufays, Long short-term memory based recurrent neural network architectures for large vocabulary speech recognition, 2014, <http://dx.doi.org/10.48550/ARXIV.1402.1128>, URL <https://arxiv.org/abs/1402.1128>.
- [19] Kyunghyun Cho, Bart van Merriënboer, Dzmitry Bahdanau, Yoshua Bengio, On the properties of neural machine translation: Encoder-decoder approaches, 2014, <http://dx.doi.org/10.48550/ARXIV.1409.1259>, URL <https://arxiv.org/abs/1409.1259>.
- [20] Jacob Devlin, Ming-Wei Chang, Kenton Lee, Kristina Toutanova, BERT: Pre-training of deep bidirectional transformers for language understanding, 2018, <http://dx.doi.org/10.48550/ARXIV.1810.04805>, URL <https://arxiv.org/abs/1810.04805>.
- [21] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, Ming Zhou, CodeBERT: A pre-trained model for programming and natural languages, 2020, <http://dx.doi.org/10.48550/ARXIV.2002.08155>, URL <https://arxiv.org/abs/2002.08155>.
- [22] S. Cao, X. Sun, L. Bo, R. Wu, B. Li, C. Tao, MVD: Memory-related vulnerability detection based on flow-sensitive graph neural networks, in: *2022 IEEE/ACM 44th International Conference on Software Engineering, ICSE, IEEE Computer Society, Los Alamitos, CA, USA, 2022*, pp. 1456–1468, <http://dx.doi.org/10.1145/3510003.3510219>, URL <https://doi.ieeecomputersociety.org/10.1145/3510003.3510219>.
- [23] David Hin, Andrey Kan, Huaming Chen, M. Ali Babar, Linevd: Statement-level vulnerability detection using graph neural networks, 2022, <http://dx.doi.org/10.48550/ARXIV.2203.05181>, URL <https://arxiv.org/abs/2203.05181>.
- [24] Yi Li, Shaohua Wang, Tien N. Nguyen, Vulnerability detection with fine-grained interpretations, in: *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, in: ESEC/FSE 2021, Association for Computing Machinery, New York, NY, USA, 2021, pp. 292–303, <http://dx.doi.org/10.1145/3468264.3468597>.
- [25] Yisroel Mirsky, George Macon, Michael Brown, Carter Yagemann, Matthew Pruett, Evan Downing, Sukarno Mertoguno, Wenke Lee, VulChecker: Graph-based vulnerability localization in source code, in: *32nd USENIX Security Symposium (USENIX Security 23)*, USENIX Association, Anaheim, CA, 2023, pp. 6557–6574, URL <https://www.usenix.org/conference/usenixsecurity23/presentation/mirsky>.
- [26] Huanting Wang, Guixin Ye, Zhanyong Tang, Shin Hwei Tan, Songfang Huang, Dingyi Fang, Yansong Feng, Lizhong Bian, Zheng Wang, Combining graph-based learning with automated data collection for code vulnerability detection, *IEEE Trans. Inf. Forensics Secur.* 16 (2021) 1943–1958, <http://dx.doi.org/10.1109/TIFS.2020.3044773>.
- [27] Yu Luo, Weifeng Xu, Dianxiang Xu, Compact abstract graphs for detecting code vulnerability with gnn models, in: *Proceedings of the 38th Annual Computer Security Applications Conference, ACSAC '22, Association for Computing Machinery, New York, NY, USA, 2022*, pp. 497–507, <http://dx.doi.org/10.1145/3564625.3564655>.
- [28] Tomas Mikolov, Kai Chen, Greg Corrado, Jeffrey Dean, Efficient estimation of word representations in vector space, 2013, <http://dx.doi.org/10.48550/ARXIV.1301.3781>, URL <https://arxiv.org/abs/1301.3781>.
- [29] Quoc V. Le, Tomas Mikolov, Distributed representations of sentences and documents, 2014, <http://dx.doi.org/10.48550/ARXIV.1405.4053>, URL <https://arxiv.org/abs/1405.4053>.
- [30] Thomas N. Kipf, Max Welling, Semi-supervised classification with graph convolutional networks, 2016, <http://dx.doi.org/10.48550/ARXIV.1609.02907>, URL <https://arxiv.org/abs/1609.02907>.
- [31] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, Yoshua Bengio, Graph attention networks, 2017, <http://dx.doi.org/10.48550/ARXIV.1710.10903>, URL <https://arxiv.org/abs/1710.10903>.
- [32] Xiao Cheng, Haoyu Wang, Jiayi Hua, Guoai Xu, Yulei Sui, DeepWukong: Statistically detecting software vulnerabilities using deep graph neural network, *ACM Trans. Softw. Eng. Methodol.* 30 (3) (2021) <http://dx.doi.org/10.1145/3436877>.
- [33] Chen Cai, Yusu Wang, A note on over-smoothing for graph neural networks, 2020, <http://dx.doi.org/10.48550/ARXIV.2006.13318>, URL <https://arxiv.org/abs/2006.13318>.
- [34] Jake Topping, Francesco Di Giovanni, Benjamin Paul Chamberlain, Xiaowen Dong, Michael M. Bronstein, Understanding over-squashing and bottlenecks on graphs via curvature, 2021, <http://dx.doi.org/10.48550/ARXIV.2111.14522>, URL <https://arxiv.org/abs/2111.14522>.
- [35] Chaitanya K. Joshi, Transformers are graph neural networks, Gradient (2021) URL <https://thegradient.pub/transformers-are-graph-neural-networks/>.
- [36] Xue Li, Yuanzhi Cheng, Understanding the message passing in graph neural networks via power iteration clustering, *Neural Netw.* 140 (2021) 130–135, <http://dx.doi.org/10.1016/j.neunet.2021.02.025>.
- [37] Guru Bhandari, Amara Naseer, Leon Moonen, CVEfixes: Automated collection of vulnerabilities and their fixes from open-source software, in: *Proceedings of the 17th International Conference on Predictive Models and Data Analytics in Software Engineering, PROMISE '21, ACM, 2021*, p. 10, <http://dx.doi.org/10.1145/3475960.3475985>.
- [38] Paul E. Black, A software assurance reference dataset: Thousands of programs with known bugs, *J. Res. Natl. Inst. Stand. Technol.* 123 (2018) <http://dx.doi.org/10.6028/jres.123.005>.

- [39] Harold Booth, Doug Rike, Gregory A. Witte, The national vulnerability database (NVD): Overview | NIST, IRL Bull. (2013) URL https://tsapps.nist.gov/publication/get_pdf.cfm?pub_id=915172.
- [40] Luana Ruiz, Fernando Gama, Alejandro Ribeiro, Gated graph recurrent neural networks, IEEE Trans. Signal Process. 68 (2020) 6303–6318, <http://dx.doi.org/10.1109/tsp.2020.3033962>.
- [41] Jeffrey Pennington, Richard Socher, Christopher Manning, GloVe: Global vectors for word representation, in: Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing, EMNLP, Association for Computational Linguistics, Doha, Qatar, 2014, pp. 1532–1543, <http://dx.doi.org/10.3115/v1/D14-1162>, URL <https://aclanthology.org/D14-1162>.
- [42] Jiahao Fan, Yi Li, Shaohua Wang, Tien N. Nguyen, A C/C++ code vulnerability dataset with code changes and CVE summaries, in: 2020 IEEE/ACM 17th International Conference on Mining Software Repositories, MSR, 2020, pp. 508–512, <http://dx.doi.org/10.1145/3379597.3387501>.
- [43] Michael Fu, Chakkrit Tantithamthavorn, LineVul: A transformer-based line-level vulnerability prediction, in: 2022 IEEE/ACM 19th International Conference on Mining Software Repositories, MSR, 2022, pp. 608–620, <http://dx.doi.org/10.1145/3524842.3528452>.
- [44] Hanjun Dai, Bo Dai, Le Song, Discriminative embeddings of latent variable models for structured data, 2016, <http://dx.doi.org/10.48550/ARXIV.1603.05629>, URL <https://arxiv.org/abs/1603.05629>.
- [45] Kaitao Song, Xu Tan, Tao Qin, Jianfeng Lu, Tie-Yan Liu, MPNet: Masked and permuted pre-training for language understanding, 2020, <http://dx.doi.org/10.48550/ARXIV.2004.09297>, URL <https://arxiv.org/abs/2004.09297>.
- [46] Wenhui Wang, Furu Wei, Li Dong, Hangbo Bao, Nan Yang, Ming Zhou, MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers, 2020, <http://dx.doi.org/10.48550/ARXIV.2002.10957>, URL <https://arxiv.org/abs/2002.10957>.
- [47] Rex Ying, Dylan Bourgeois, Jiaxuan You, Marinka Zitnik, Jure Leskovec, GNNExplainer: Generating explanations for graph neural networks, 2019, <http://dx.doi.org/10.48550/ARXIV.1903.03894>, URL <https://arxiv.org/abs/1903.03894>.
- [48] Victor Sanh, Lysandre Debut, Julien Chaumond, Thomas Wolf, DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter, 2019, <http://dx.doi.org/10.48550/ARXIV.1910.01108>, URL <https://arxiv.org/abs/1910.01108>.
- [49] Zhenzhong Lan, Mingda Chen, Sebastian Goodman, Kevin Gimpel, Piyush Sharma, Radu Soricut, ALBERT: A lite BERT for self-supervised learning of language representations, 2019, <http://dx.doi.org/10.48550/ARXIV.1909.11942>, URL <https://arxiv.org/abs/1909.11942>.
- [50] Iz Beltagy, Matthew E. Peters, Arman Cohan, Longformer: The long-document transformer, 2020, <http://dx.doi.org/10.48550/ARXIV.2004.05150>, URL <https://arxiv.org/abs/2004.05150>.
- [51] Zhilin Yang, Zihang Dai, Yiming Yang, Jaime Carbonell, Ruslan Salakhutdinov, Quoc V. Le, XLNet: Generalized autoregressive pretraining for language understanding, 2019, <http://dx.doi.org/10.48550/ARXIV.1906.08237>, URL <https://arxiv.org/abs/1906.08237>.
- [52] Armand Joulin, Edouard Grave, Piotr Bojanowski, Matthijs Douze, Herve Jégou, Tomas Mikolov, FastText.zip: Compressing text classification models, 2016, <http://dx.doi.org/10.48550/ARXIV.1612.03651>, URL <https://arxiv.org/abs/1612.03651>.
- [53] f. Bagheri, Péter Hegedűs, A comparison of different source code representation methods for vulnerability prediction in python, in: Communications in Computer and Information Science, Springer International Publishing, 2021, pp. 267–281, http://dx.doi.org/10.1007/978-3-030-85347-1_20.
- [54] Xinying Song, Alex Salcianu, Yang Song, Dave Dopson, Denny Zhou, Fast WordPiece tokenization, 2020, <http://dx.doi.org/10.48550/ARXIV.2012.15524>, URL <https://arxiv.org/abs/2012.15524>.
- [55] Jimmy Lei Ba, Jamie Ryan Kiros, Geoffrey E. Hinton, Layer normalization, 2016, <http://dx.doi.org/10.48550/ARXIV.1607.06450>, URL <https://arxiv.org/abs/1607.06450>.
- [56] Jesse Vig, A multiscale visualization of attention in the transformer model, in: Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics: System Demonstrations, Association for Computational Linguistics, Florence, Italy, 2019, pp. 37–42, <http://dx.doi.org/10.18653/v1/P19-3007>, URL <https://www.aclweb.org/anthology/P19-3007>.
- [57] Ilya Loshchilov, Frank Hutter, Decoupled weight decay regularization, 2017, <http://dx.doi.org/10.48550/ARXIV.1711.05101>, URL <https://arxiv.org/abs/1711.05101>.
- [58] Kevin Clark, Urvashi Khandelwal, Omer Levy, Christopher D. Manning, What does BERT look at? An analysis of bert's attention, 2019, <http://dx.doi.org/10.48550/ARXIV.1906.04341>, URL <https://arxiv.org/abs/1906.04341>.
- [59] Nils Reimers, Iryna Gurevych, Making monolingual sentence embeddings multilingual using knowledge distillation, 2020, <http://dx.doi.org/10.48550/ARXIV.2004.09813>, URL <https://arxiv.org/abs/2004.09813>.
- [60] Hamel Husain, Ho-Hsiang Wu, Tiferet Gazit, Miltiadis Allamanis, Marc Brockschmidt, CodeSearchNet challenge: Evaluating the state of semantic code search, 2019, <http://dx.doi.org/10.48550/ARXIV.1909.09436>, URL <https://arxiv.org/abs/1909.09436>.
- [61] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, Illia Polosukhin, Attention is all you need, 2017, <http://dx.doi.org/10.48550/ARXIV.1706.03762>, URL <https://arxiv.org/abs/1706.03762>.
- [62] Nikita Kitaev, Lukasz Kaiser, Anselm Levskaya, Reformer: The efficient transformer, 2020, <http://dx.doi.org/10.48550/ARXIV.2001.04451>, URL <https://arxiv.org/abs/2001.04451>.
- [63] Andrew Jaegle, Felix Gimeno, Andrew Brock, Andrew Zisserman, Oriol Vinyals, Joao Carreira, Perceiver: General perception with iterative attention, 2021, <http://dx.doi.org/10.48550/ARXIV.2103.03206>, URL <https://arxiv.org/abs/2103.03206>.
- [64] Albert Gu, Karan Goel, Christopher Ré, Efficiently modeling long sequences with structured state spaces, in: The International Conference on Learning Representations, ICLR, 2022.



Hoai-Chau Tran is a final-year undergraduate student majoring in computer science at the VNUHCM - University of Science. Currently, he is working as a research assistant at the AISIA research lab. His research interest includes machine learning, deep learning for natural language processing, and multi-modal data analytics.



Anh-Duy Tran is a Doctoral Researcher in the research group DistriNet at the Computer Science Department of the Katholieke Universiteit Leuven (KU Leuven), Belgium. His research focuses include various subjects in Computer and System Security, mostly in Software Security, Computer and Multimedia Forensics, Mobile Security, Malware Analysis, and Privacy. He obtained the Diplôme d'ingénieur in Communication System Security at EURECOM and Télécom Paris, France, in 2019. He founded the Computer Security Club at the University of Science, VNU-HCM. He sometimes plays CTF with the codename *pakkunandy*, and he also hosts HCMUS-CTF every year.



Kim-Hung Le (Member, IEEE) was born in Ho Chi Minh City, Vietnam, in 1990. He received his B.S. degree in networking engineering from the University of Information Technology—Vietnam National University in 2013, his M.S. degree in network security from Telecom ParisTech, France, in 2016, and his Ph.D. degree in the Internet of Things from Sorbonne University, France, in 2019. Currently, he is a Lecturer in the Data Communication and Networking Department at the University of Information Technology—Vietnam National University. His research interests include applying deep learning and artificial intelligence to the Internet of Things, data cleaning, and developing energy-efficient methods for embedded devices.